

ESP32中文学习资料

1.前言

最近博主听到QQ群里面问得比较多的问题：
“博哥，有玩过OLED吗？”
“博哥，有试过在ESP8266上调成功过SSD1306吗？”
“博哥，OLED上显示天气信息怎么弄？”
.....

诸如此类的问题，在博主看来，无非都是对OLED库用法的不熟悉甚至不了解。Arduino OLED库众多，博主也曾经介绍过一款 《博哥OLED系列》- 玩转SSD1306-12864 OLED Adafruit_GFX 和 Adafruit_SSD1306库。但是，博主今天要介绍的是目前Arduino平台上使用最广泛的OLED库 —— U8G2库，也是很多初学者容易懵逼的一个第三方库。目前在github上超过1K star，1800次commit，可以说维护热度很高，读者大可放心使用。

至于这个库为什么这么火爆，请读者认真看完博文自行评价，博主只能说功能真的很好很强大！！！！

那么，博主首先立个flag，阅读完本篇博文的最终效果——读者以后在Arduino平台上开发OLED项目的时候，可以考虑U8G2库，并且知道U8G2库的具体用法，甚至可以能够当做工具书来翻阅。

进入正文之前，博主要给读者灌输两个非常重要的知识点：

- 其一 像素点点阵

OLED其实就是一个M x n 的像素点阵，想显示什么就得把具体位置的像素点亮起来。对于每一个像素点，有可能是1点亮，也有可能是0点亮；

- 其二 坐标系



在坐标系中，左上角是原点，向右是X轴，向下是Y轴。

希望读者能仔细理解这两个知识点。

2.U8g2简介

2.1 U8g2是什么

U8g2是嵌入式设备的单色图形库，一句话简单明了。主要应用于嵌入式设备，包括我们常见的单片机；

2.2 U8g2支持的显示控制器

U8g2支持单色OLED和LCD，包括以下控制器：SSD1305，SSD1306，SSD1309，SSD1322，SSD1325，SSD1327，SSD1329，SSD1606，SSD1607，SH1106，SH1107，SH1108，SH1122，T6963，RA8835，LC7981，PCD8544，PCF8812，HX1230，UC1601，UC1604，UC1608，UC1610，UC1611，UC1701，ST7565，ST7567，ST7588，ST75256，NT7534，IST3020，ST7920，LD7032，KS0108，SED1520，SBN1661，IL3820，MAX7219（有关完整列表，请参见 [此处](#)）。

可以说，基本上主流的显示控制器都支持，比如我们常见的SSD1306 12864，读者在使用该库之前请查阅自己的OLED显示控制器是否处于支持列表中。

2.2 U8g2支持的Arduino主板

可以说基本上所有Arduino API的主板都得到U8g2的支持。包括：

- Aruino Zero，Uno，Mega，Due，101，MKR Zero以及所有其他Arduino官方主板
- 基于Arduino平台的STM32
- 基于Arduino平台的ESP8266和ESP32
- 甚至其他不知名的基于Arduino平台的开发板

所以说，读者完全不用担心兼容性问题，放心使用。

2.3 U8g2如何在Arduino平台上安装

Arduino库U8g2可以从Arduino IDE的库管理器安装，读者在库管理器搜索“U8g2”关键字就可以下载安装：

类型 全部 主题 全部 U8g2

LCDMenuLib2 by Nils Feldkaemper

Easy creation of a tree based menu with screensaver and multi layers. Examples for the basic function and different output types [serial monitor, liquidcrystal, i2c, graphic displays (u8glib / u8g2lib...)]

[More info](#)

U8g2 by oliver 版本 2.24.3 INSTALLED

Monochrome LCD, OLED and eInk Library. Display controller: SSD1305, SSD1306, SSD1309, SSD1322, SSD1325, SSD1327, SSD1329, SSD1606, SSD1607, SH1106, SH1107, SH1108, SH1122, T6963, RA8835, LC7981, PCD8544, PCF8812, HX1230, UC1601, UC1604, UC1608, UC1610, UC1611, UC1701, ST7565, ST7567, ST7588, ST75256, NT7534, IST3020, ST7920, LD7032, KS0108, SED1520, SBN1661, IL3820, MAX7219. Interfaces: I2C, SPI, Parallel. Monochrome LCD, OLED and eInk Library. Successor of U8glib. Supported display controller: SSD1305, SSD1306, SSD1309, SSD1322, SSD1325, SSD1327, SSD1329, SSD1606, SSD1607, SH1106, SH1107, SH1108, SH1122, T6963, RA8835, LC7981, PCD8544, PCF8812, HX1230, UC1601, UC1604, UC1608, UC1610, UC1611, UC1701, ST7565, ST7567, ST7588, ST75256, NT7534, IST3020, ST7920, LD7032, KS0108, SED1520, SBN1661, IL3820, MAX7219. Supported interfaces: I2C, SPI, Parallel. Features: UTF8, >700 fonts, U8x8 char output.

[More info](#)

选择版本

安装

更新

U8g2_for_Adafruit_GFX by oliver

Add U8g2 fonts to any Adafruit GFX based graphics library. Use our favorite Adafruit graphics library together with fonts from U8g2 project (<https://github.com/olikraus/u8g2/wiki/fntlistall>).

[More info](#)

下载完毕，测试一下库是否安装成功：

```
#include <U8g2lib.h>
void setup() {
    // put your setup code here, to run once:
}

void loop() {
    // put your main code here, to run repeatedly:
}
```

编译成功，证明你本地已经加载了U8G2库。

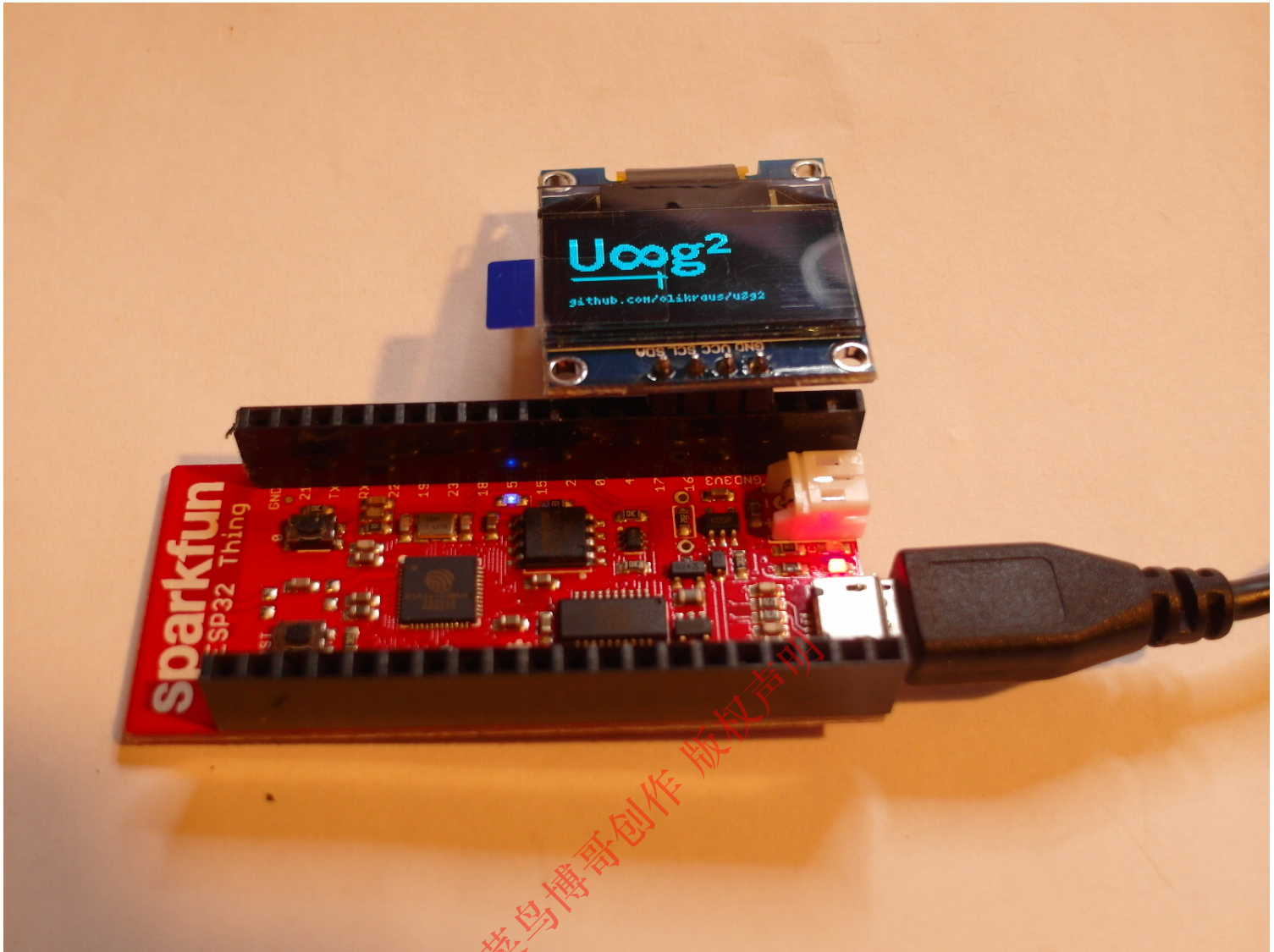
2.4 U8g2的优势

为什么要运用U8g2库？也就是说U8g2库能带给我们什么样的开发便利。在博主看来，主要考虑几个方面：

- U8g2库平台支持性好，基本上支持绝大部分Arduino开发板，特别也博主比较喜欢的ESP8266；
- U8g2库显示控制器支持性好，基本市面上的OLED都完美支持；
- U8g2库 API众多，特别支持了中文，支持了不同字体，这是一个对于开发者来说不小的福利。

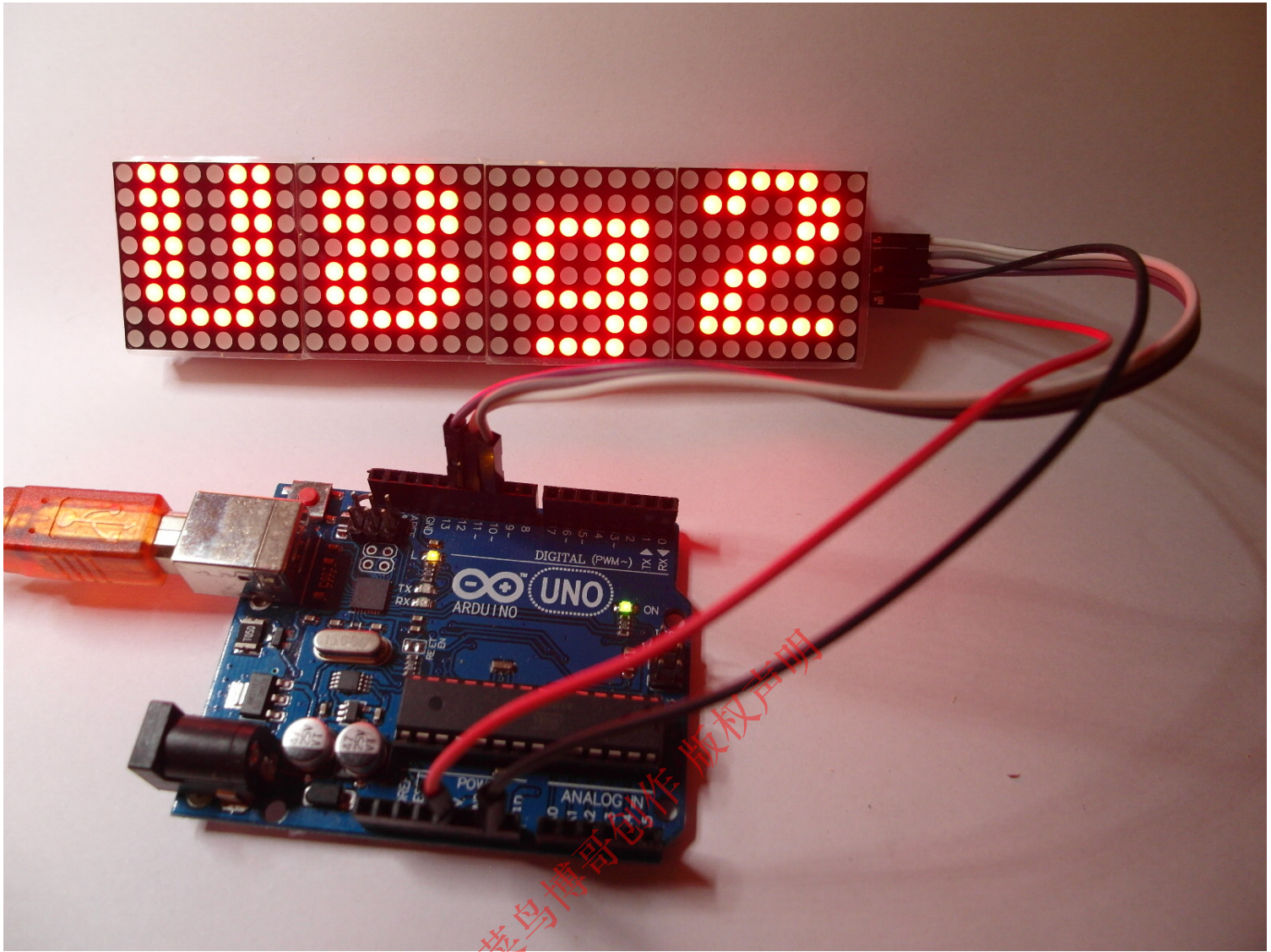
以下是官方提供的一些[U8G2库测试图](#)，博主挑选了几张常见的以及附上U8G2库的配置（读者可以先不用理会配置，等看完博文再次阅读就会了解含义）：

- ESP32 and SSD1306 OLED



```
U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R2, /* clock=*/ 16, /* data=*/ 17, /* reset=*/  
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C  
U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R2, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/  
17); // ESP32 Thing, HW I2C with pin remapping
```

- MAX7219 32x8 LED Matrix



```
U8G2_MAX7219_32X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/  
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
```

- SSD1305 128x32



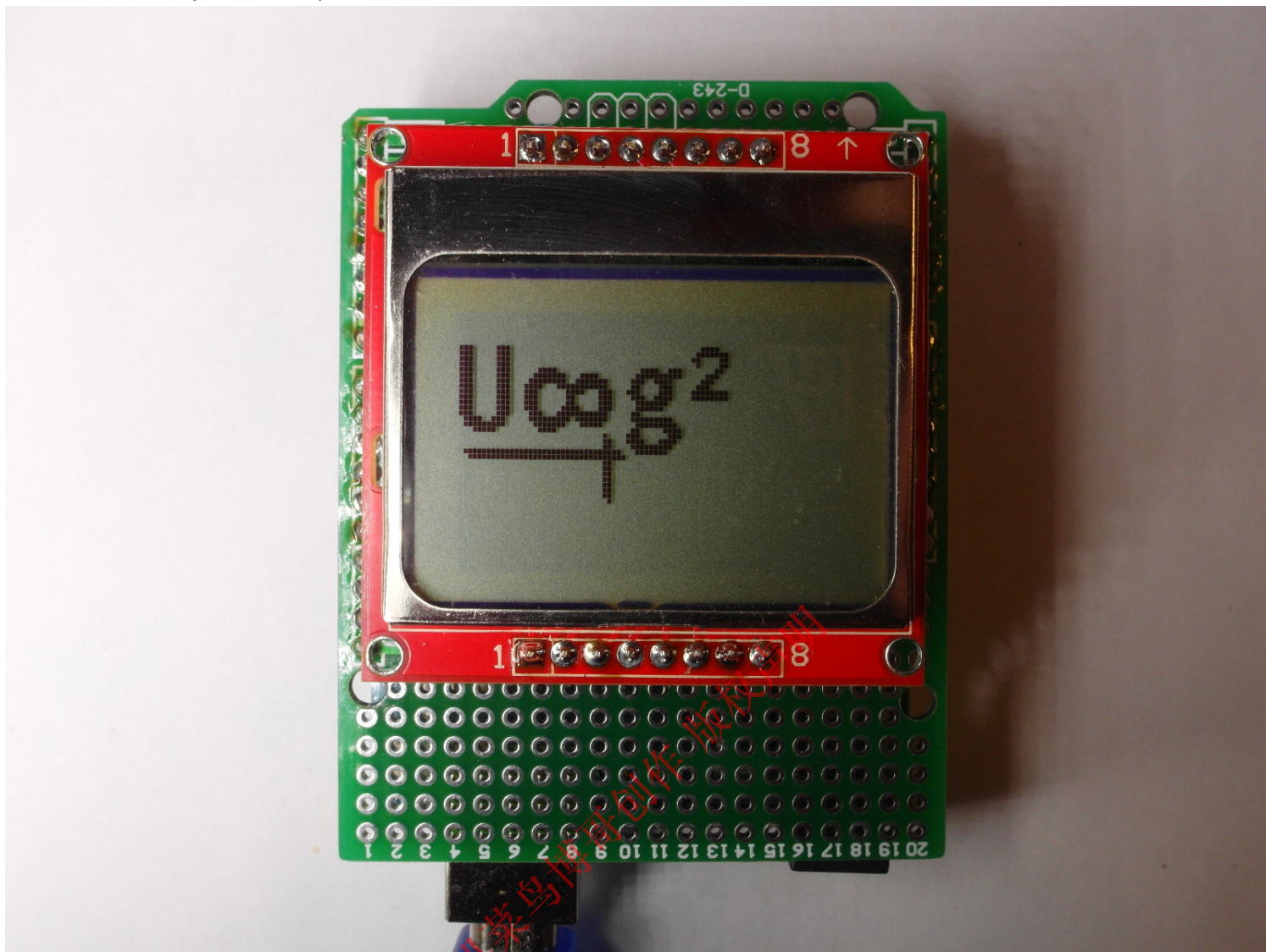
```
U8G2_SSD1305_128X32_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
```



- SSD1306 128x32

```
U8X8_SSD1306_128X32_UNIVISION_SW_I2C u8x8(/* clock=A5*/ 19, /* data=A4*/ 18);
```

- PCD8544 84x48 (Nokia 5110) LCD



```
U8G2_PCD8544_84X48_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*  
reset=*/ 8);    // Nokia 5110 Display
```

- ST7920 128x64 LCD in 8080 parallel mode



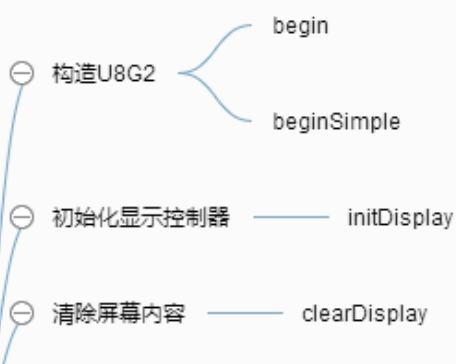
```
U8G2_ST7920_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/  
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
```

这只是一些常见的显示器，可以说，已经足够覆盖到我们常见的范围了。

3.U8g2库详解

3.1 U8g2库百度脑图

老规则，先上一个图：



基本函数

- 是否开启省电模式 ———— `setPowerSave`
- 清除操作 ———— `clear`
- 清除缓冲区 ———— `clearBuffer`
- 禁用 UTF8打印 ———— `disableUTF8Print`
- 启用 UTF8打印 ———— `enableUTF8Print`
- 重置显示光标的位置 ———— `home`

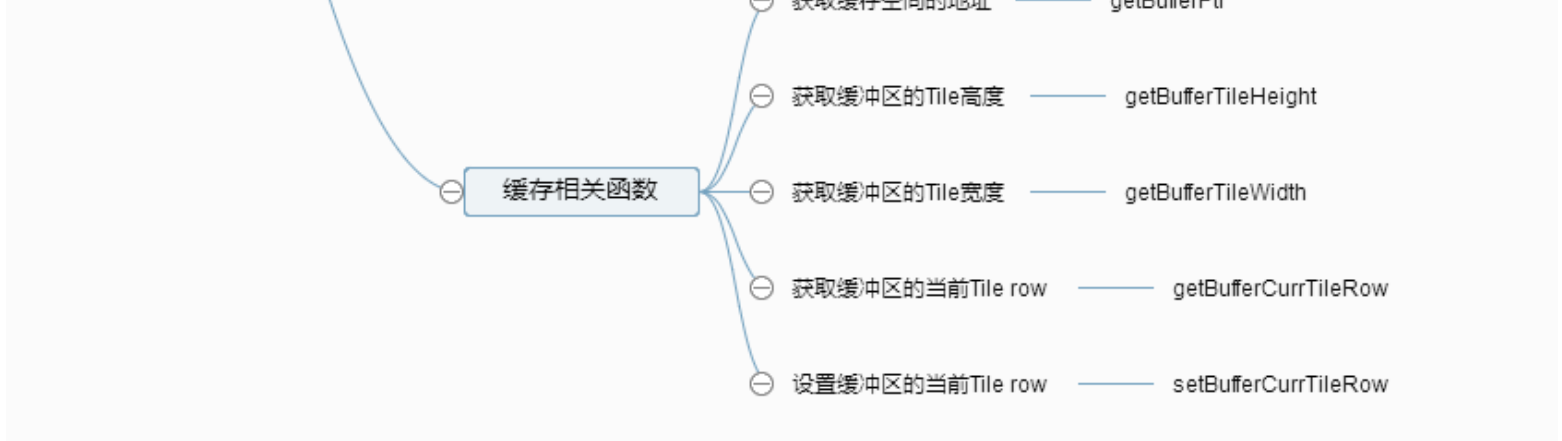
绘制相关函数

- 画实心方形 ———— `drawBox`
- 画空心圆 ———— `drawCircle`
- 画实心圆 ———— `drawDisc`
- 画空心椭圆 ———— `drawEllipse`
- 画实心椭圆 ———— `drawFilledEllipse`
- 画空心方形 ———— `drawFrame`
- 绘制字体字集的符号 ———— `drawGlyph`
- 绘制水平线 ———— `drawHLine`
- 两点之间绘制线 ———— `drawLine`
- 绘制像素点 ———— `drawPixel`
- 绘制圆角实心方形 ———— `drawRBox`
- 绘制圆角空心方形 ———— `drawRFrame`
- 绘制字符串 ———— `drawStr`
- 绘制实心三角形 ———— `drawTriangle`
- 绘制UTF8编码的字符 ———— `drawUTF8`
- 绘制竖直线 ———— `drawVLine`

U8G2库

显示配置相关函数

- 绘制图像 — drawXBM()/drawXBMP()
- 分页绘制命令 — firstPage()/nextPage()
- 绘制内容 — print
- 绘制缓冲区的内容 — sendBuffer
- 获取基准线以上的高度 — getAscent
- 获取基准线以下的高度 — getDescent
- 获取显示器的高度 — getDisplayHeight
- 获取显示器的宽度 — getDisplayWidth
- 获取当前字体里的最大字符的高度 — getMaxCharHeight
- 获取当前字体里的最大字符的宽度 — getMaxCharWidth
- 获取字符串的像素宽度 — getStrWidth
- 获取UTF-8字符串的像素宽度 — getUTF8Width
- 设置自动清除缓冲区 — setAutoPageClear
- 设置位图模式 — setBitmapMode
- 设置总线时钟 — setBusClock
- 设置采集窗口大小 — setClipWindow
- 设置绘制光标位置 — setCursor
- 设置显示器的旋转角度 — setDisplayRotation
- 设置绘制颜色 — setDrawColor
- 设置字体集 — setFont
- 设置字体方向 — setFontDirection
- 获取缓存空间的地址 — getBufferPtr



方法可以分为四大类：

- 基本函数
- 绘制相关函数
- 显示配置相关函数
- 缓存相关函数

3.2 U8g2库函数详解

3.2.1 基本函数

3.2.1.1 u8g2.begin() —— 构造U8G2

函数说明：

```
/**
 * 初始化U8g2库
 * @Note 关联方法 initDisplay clearDisplay setPowerSave
 */
bool U8G2::begin(void)
```

源码说明：

```
bool begin(void) {
    /* note: call to u8x8_utf8_init is not required here, this is done in the setup procedures before */
    initDisplay(); //初始化显示器
    clearDisplay(); // 重置清屏
    setPowerSave(0); //唤醒屏幕
    return 1;
}
```

3.2.1.2 u8g2.beginSimple() —— 构造U8G2

函数说明：

```
/**
 * 简单初始化U8g2库
 * @Note 关联方法 beginSimple
```

```
*/  
void U8G2::beginSimple(void);
```

源码说明：

```
void beginSimple(void) {  
  /* does not clear the display and does not wake up the display */  
  /* user is responsible for calling clearDisplay() and setPowerSave(0) */  
  initDisplay(); //初始化显示器  
}
```

注意点：

- 读者可以看到和begin()函数的区别，需要用户自行控制初始化过程，给了一定的自由度，不过博主建议大家还是直接用begin函数吧。

3.2.1.3 u8g2.initDisplay() —— 初始化显示控制器

函数说明：

```
/**  
 * 初始化显示控制器  
 */  
void U8G2::initDisplay(void)
```

注意点：

- 这个方法不需要我们单独调用，会在begin函数主动调用一次，我们主要理解即可，会在里面针对具体的OLED进行配置；

3.2.1.4 u8g2.clearDisplay() —— 清除屏幕内容

函数说明：

```
/**  
 * 清除屏幕  
 */  
void U8G2::clearDisplay(void)
```

注意点：

- 这个方法不需要我们单独调用，会在begin函数主动调用一次，我们主要理解即可；
- 不要在 firstPage 和 nextPage 函数之间调用该方法；

3.2.1.5 u8g2.setPowerSave() —— 是否开启省电模式

函数说明：

```
/**  
 * 清除显示缓冲区  
 * @param is_enable
```

```

*      1 表示启用显示器的省电模式，屏幕上看不到任何东西
*
*      0 表示禁用省电模式
*/
void U8G2::setPowerSave(uint8_t is_enable)

```

注意点：

- 不管是启用还是禁用，显示器需要的内存消耗是不会变的，说到底就是为了关闭屏幕，做到省电；
- 所以这里就可以理解为什么初始化需要 setPowerSave(0);

3.2.1.6 u8g2.clear() —— 清除操作

函数说明：

```

/**
 * 清除屏幕显示，清除缓冲区，光标回到左上角原点位置 (0,0)
 * @Note 关联方法 home clearDisplay clearBuffer
 */
void U8G2::clear(void)

```

源码说明：

```

void clear(void) {
    home(); //回到原点
    clearDisplay(); //清除屏幕上的显示
    clearBuffer(); //清除缓冲区
}

```

3.2.1.7 u8g2.clearBuffer() —— 清除缓冲区

函数说明：

```

/**
 * 清除内存中数据缓冲区
 */
void U8G2::clearBuffer(void)

```

注意点：

- 一般这个函数是与sendBuffer函数配对使用，通常用法如下：

```

void loop(void) {
    u8g2.clearBuffer();
    // ... write something to the buffer
    u8g2.sendBuffer();
    delay(1000);
}

```

3.2.1.8 u8g2.disableUTF8Print() —— 禁用 UTF8打印

函数说明：

```
/**
 * 禁用Arduino平台下支持输出UTF8字符集，默认是开启
 */
void U8G2::disableUTF8Print(void)
```

3.2.1.9 u8g2.enableUTF8Print() —— 启用 UTF8打印

函数说明：

```
/**
 * 开启Arduino平台下支持输出UTF8字符集
 */
void U8G2::enableUTF8Print(void)
```

注意点：

- 我们的中文字符就是UTF8；
- 常见例子

```
void setup(void) {
    u8g2.begin();
    u8g2.enableUTF8Print();           // enable UTF8 support for the Arduino print() function
}
void loop(void) {
    u8g2.setFont(u8g2_font_unifont_t_chinese2); // use chinese2 for all the glyphs of "你好世界"
    u8g2.firstPage();
    do {
        u8g2.setCursor(0, 40);
        u8g2.print("你好世界");      // Chinese "Hello World"
    } while ( u8g2.nextPage() );
    delay(1000);
}
```

3.2.1.10 u8g2.home() —— 重置显示光标的位置

函数说明：

```
/**
 * 重置显示光标的位置，回到原点 (0, 0)
 * @Note 关联方法 print clear
 */
void U8G2::home(void)
```

3.2.2 绘制相关函数

3.2.2.1 u8g2.drawBox() —— 画实心方形

函数说明：

```

/**
 * 画实心方形，左上角坐标为(x,y)，宽度为w，高度为h
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param w 方形的宽度
 * @param h 方形的高度
 * @Note 关联方法 drawFrame setDrawColor
 */
void U8G2::drawBox(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w, u8g2_uint_t h)

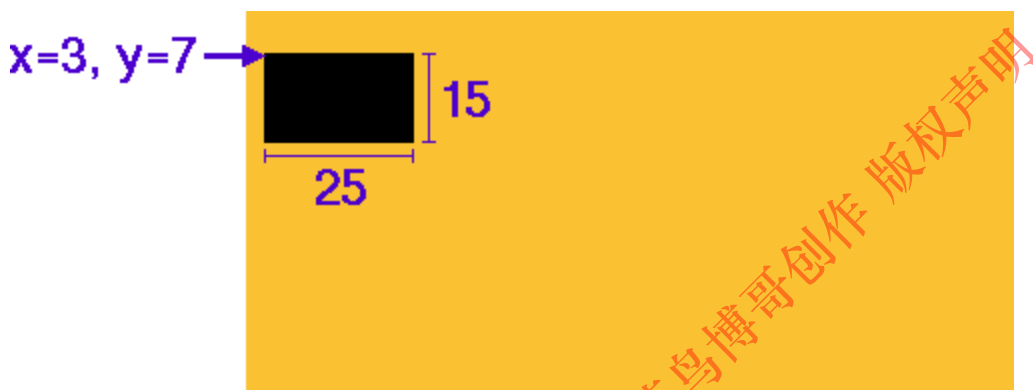
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；

示例：

```
u8g2.drawBox(3, 7, 25, 15);
```



3.2.2.2 u8g2.drawCircle() —— 画空心圆

函数说明：

```

/**
 * 画空心圆，圆心坐标为(x0,y0)，半径为rad
 * @param x0 圆点的x坐标
 * @param y0 圆点的y坐标
 * @param rad 圆形的半径
 * @param opt 圆形选项
 *         U8G_DRAW_ALL 整个圆
 *         U8G2_DRAW_UPPER_RIGHT 右上部分的圆弧
 *         U8G2_DRAW_UPPER_LEFT 左上部分的圆弧
 *         U8G2_DRAW_LOWER_LEFT 左下部分的圆弧
 *         U8G2_DRAW_LOWER_RIGHT 右下部分的圆弧
 *         选项可以通过 | 操作符来组合
 * @Note 关联方法 drawDisc setDrawColor
 */
void U8G2::drawCircle(u8g2_uint_t x0, u8g2_uint_t y0, u8g2_uint_t rad, uint8_t opt = U8G2_DRAW_ALL)

```

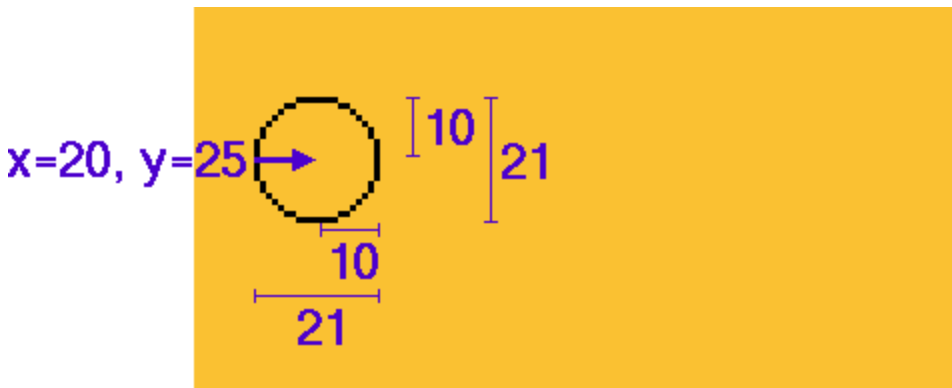
注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；

- 直径等于 $2rad + 1$ ；

示例：

```
u8g2.drawCircle(20, 25, 10, U8G2_DRAW_ALL);
```



3.2.2.3 u8g2.drawDisc() —— 画实心圆

函数说明：

```
/**
 * 画实心圆，圆心坐标为(x0, y0)，半径为rad
 * @param x0 圆点的x坐标
 * @param y0 圆点的y坐标
 * @param rad 圆形的半径
 * @param opt 圆形选项
 *      U8G_DRAW_ALL 整个圆
 *      U8G2_DRAW_UPPER_RIGHT 右上部分的圆弧
 *      U8G2_DRAW_UPPER_LEFT 左上部分的圆弧
 *      U8G2_DRAW_LOWER_LEFT 左下部分的圆弧
 *      U8G2_DRAW_LOWER_RIGHT 右下部分的圆弧
 * 选项可以通过 | 操作符来组合
 * @Note 关联方法 drawCircle setDrawColor
 */
void U8G2::drawDisc(u8g2_uint_t x0, u8g2_uint_t y0, u8g2_uint_t rad, uint8_t opt = U8G_DRAW_ALL)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；
- 直径等于 $2rad + 1$ ；

3.2.2.4 u8g2.drawEllipse() —— 画空心椭圆

函数说明：

```
/**
 * 画空心椭圆，圆心坐标为(x0, y0)，半径为rad
 * @param x0 圆点的x坐标
 * @param y0 圆点的y坐标
 * @param rx 椭圆形水平x方向的半径
 * @param ry 椭圆形竖直y方向的半径
```

```

* @param opt 圆形选项
*      U8G_DRAW_ALL 整个椭圆
*      U8G2_DRAW_UPPER_RIGHT 右上部分的圆弧
*      U8G2_DRAW_UPPER_LEFT 左上部分的圆弧
*      U8G2_DRAW_LOWER_LEFT 左下部分的圆弧
*      U8G2_DRAW_LOWER_RIGHT 右下部分的圆弧
*      选项可以通过 | 操作符来组合
* @Note 关联方法 drawCircle
*/
void U8G2::drawEllipse(u8g2_uint_t x0, u8g2_uint_t y0, u8g2_uint_t rx, u8g2_uint_t ry, uint8_t opt)

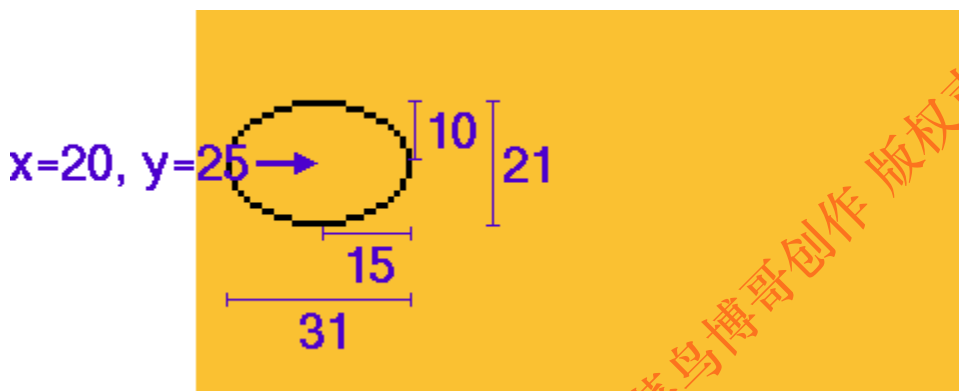
```

注意点：

- rx*ry 在8位模式的u8g2必须小于512（博主暂且没有理解）；

示例：

```
u8g2.drawEllipse(20, 25, 15, 10, U8G2_DRAW_ALL);
```



3.2.2.5 u8g2.drawFilledEllipse() —— 画实心椭圆

函数说明：

```

/**
 * 画实心椭圆，圆心坐标为(x0,y0)，半径为rad
 * @param x0 圆点的x坐标
 * @param y0 圆点的y坐标
 * @param rx 椭圆形水平x方向的半径
 * @param ry 椭圆形竖直y方向的半径
 * @param opt 圆形选项
 *      U8G_DRAW_ALL 整个椭圆
 *      U8G2_DRAW_UPPER_RIGHT 右上部分的圆弧
 *      U8G2_DRAW_UPPER_LEFT 左上部分的圆弧
 *      U8G2_DRAW_LOWER_LEFT 左下部分的圆弧
 *      U8G2_DRAW_LOWER_RIGHT 右下部分的圆弧
 *      选项可以通过 | 操作符来组合
 * @Note 关联方法 drawCircle
 */
void U8G2::drawFilledEllipse(u8g2_uint_t x0, u8g2_uint_t y0, u8g2_uint_t rx, u8g2_uint_t ry, uint8_t opt)

```

注意点：

- `rx*ry` 在8位模式的u8g2必须小于512（博主暂且没有理解）；

3.2.2.6 u8g2.drawFrame() —— 画空心方形

函数说明：

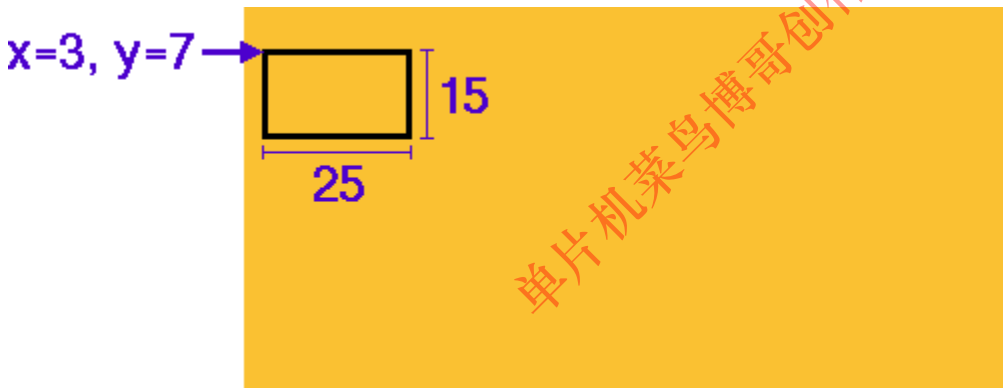
```
/**
 * 画空心方形，左上角坐标为(x,y)，宽度为w，高度为h
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param w 方形的宽度
 * @param h 方形的高度
 * @Note 关联方法 setDrawColor
 */
void U8G2::drawFrame(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w, u8g2_uint_t h)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由`setDrawColor`设置；

示例：

```
u8g2.drawFrame(3,7,25,15);
```



3.2.2.7 u8g2.drawGlyph() —— 绘制字体字集的符号

函数说明：

```
/**
 * 绘制字体字集里面定义的符号
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param encoding 字符的unicode值
 * @Note 关联方法 setFont
 */
void U8G2::drawGlyph(u8g2_uint_t x, u8g2_uint_t y, uint16_t encoding)
```

注意点：

- U8g2支持16位以内的unicode字符集，也就是说encoding的范围为0-65535，`drawGlyph`方法只能绘制存在于所使用的字体字集中的unicode值；

- 这个绘制方法依赖于当前的字体模式和绘制颜色；

3.2.2.8 u8g2.drawHLine() —— 绘制水平线

函数说明：

```
/**
 * 绘制水平线
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param w 水平线的长度
 * @Note 关联方法 setDrawColor
 */
void U8G2::drawHLine(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；

3.2.2.9 u8g2.drawLine() —— 两点之间绘制线

函数说明：

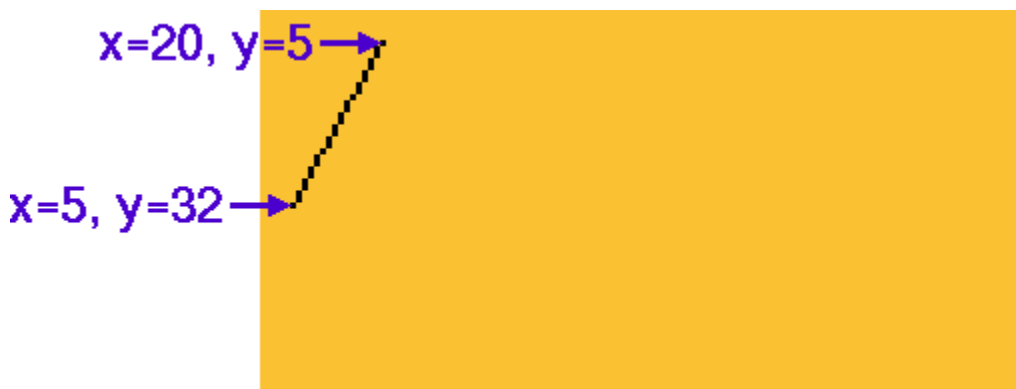
```
/**
 * 绘制线，从坐标(x0, y0) 到(x1, y1)
 * @param x0 端点0的x坐标
 * @param y0 端点0的y坐标
 * @param x1 端点1的x坐标
 * @param y1 端点1的y坐标
 * @Note 关联方法 setDrawColor
 */
void U8G2::drawLine(u8g2_uint_t x0, u8g2_uint_t y0, u8g2_uint_t x1, u8g2_uint_t y1)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；

示例：

```
u8g2.drawLine(20, 5, 5, 32);
```



3.2.2.10 u8g2.drawPixel() —— 绘制像素点

函数说明：

```
/**
 * 绘制像素点，坐标(x,y)
 * @param x 像素点的x坐标
 * @param y 像素点的y坐标
 * @Note 关联方法 setDrawColor
 */
void U8G2::drawPixel(u8g2_uint_t x, u8g2_uint_t y)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；
- 你会发现很多绘制方法的底层都是调用drawPixel，毕竟像素属于最小颗粒度；
- 我们可以利用这个绘制方法自定义自己的图形显示；

3.2.2.11 u8g2.drawRBox() —— 绘制圆角实心方形

函数说明：

```
/**
 * 绘制圆角实心方形，左上角坐标为(x,y)，宽度为w，高度为h，圆角半径为r
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param w 方形的宽度
 * @param h 方形的高度
 * @param r 圆角半径
 */
void U8G2::drawRBox(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w, u8g2_uint_t h, u8g2_uint_t r)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；
- 要求， $w \geq 2*(r+1)$ 并且 $h \geq 2*(r+1)$ ，这是显而易见的限制；

3.2.2.12 u8g2.drawRFrame() —— 绘制圆角空心方形

函数说明：

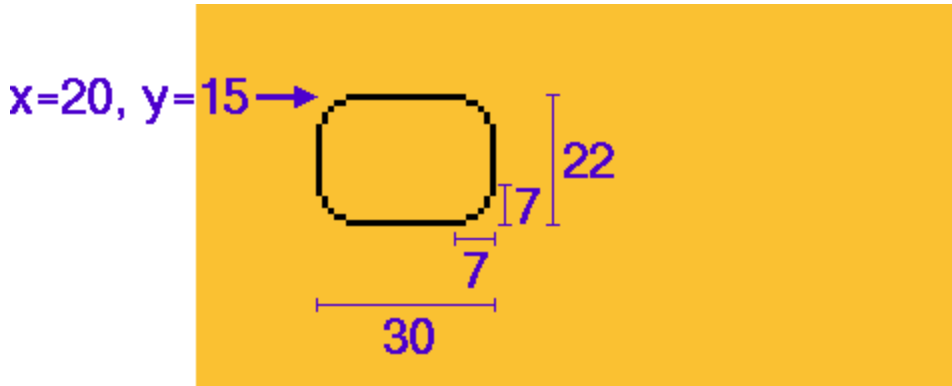
```
/**
 * 绘制圆角空心方形，左上角坐标为(x,y)，宽度为w，高度为h，圆角半径为r
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param w 方形的宽度
 * @param h 方形的高度
 * @param r 圆角半径
 */
void U8G2::drawRFrame(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w, u8g2_uint_t h, u8g2_uint_t r)
```

注意点：

- 如果支持绘制颜色（也就是不是单色显示器），那么由setDrawColor设置；
- 要求， $w \geq 2*(r+1)$ 并且 $h \geq 2*(r+1)$ ，这是显而易见的限制

示例：

```
u8g2.drawRFrame(20,15,30,22,7);
```



3.2.2.13 u8g2.drawStr() —— 绘制字符串

函数说明：

```
/**
 * 绘制字符串
 * @param x 左上角的x坐标
 * @param y 左上角的y坐标
 * @param s 绘制字符串内容
 * @return 字符串的长度
 */
u8g2_uint_t U8g2::drawStr(u8g2_uint_t x, u8g2_uint_t y, const char *s)
```

注意点：

- 需要先设置字体，调用setFont方法；
- 这个方法不能绘制encoding超过256的，超过256需要用drawUTF8或者drawGlyph；说白了就是一般用来显示英文字符；
- x, y属于字符串左下角的坐标；

示例：

```
u8g2.setFont(u8g2_font_ncenB14_tr);
u8g2.drawStr(0,15,"Hello World!");
```



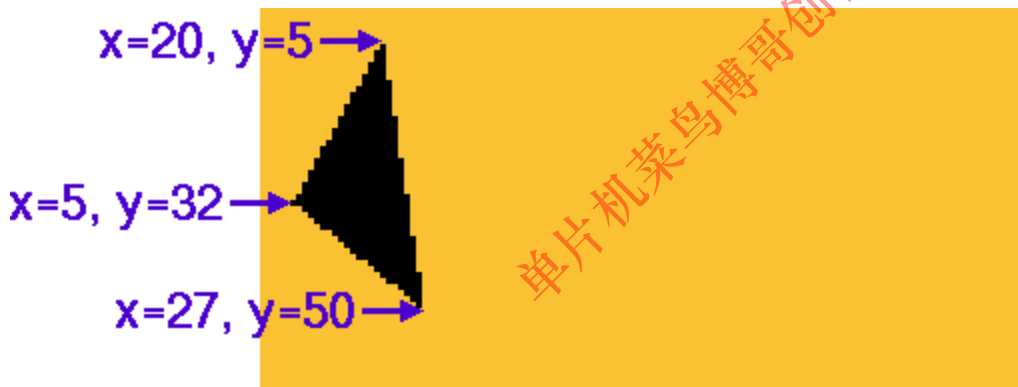
3.2.2.14 u8g2.drawTriangle() —— 绘制实心三角形

函数说明：

```
/**
 * 绘制实心三角形，定点坐标分别为(x0, y0), (x1, y1), (x2, y2)
 */
void U8G2::drawTriangle(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2)
```

示例：

```
u8g2.drawTriangle(20,5, 27,50, 5,32);
```



3.2.2.15 u8g2.drawUTF8() —— 绘制UTF8编码的字符

函数说明：

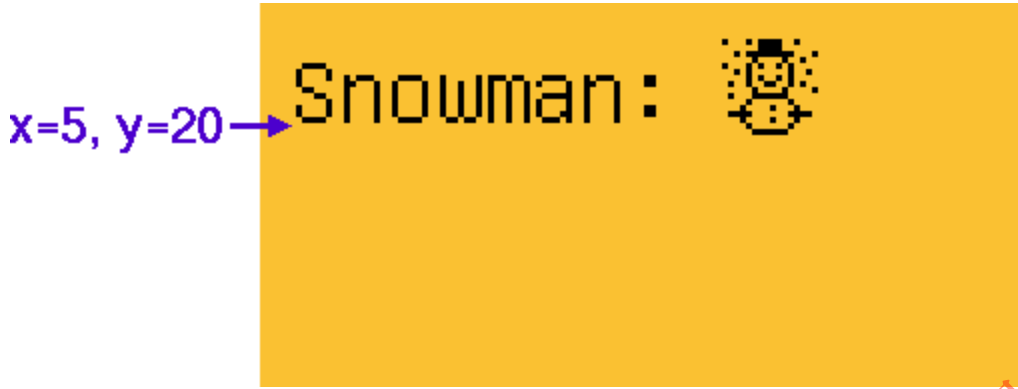
```
/**
 * 绘制UTF8编码的字符串
 * @param x 字符串在屏幕上的左下角x坐标
 * @param y 字符串在屏幕上的左下角y坐标
 * @param s 需要绘制的UTF-8编码字符串
 * @return 返回字符串的长度
 */
u8g2_uint_t U8g2::drawUTF8(u8g2_uint_t x, u8g2_uint_t y, const char *s)
```

注意点：

- 使用该方法，有两个前提。首先是你的编译器需要支持UTF-8编码，对于绝大部分Arduino板子已经支持；其次，显示的字符串需要存为“UTF-8”编码，Arduino IDE上默认支持；
- 该方法需要依赖于fontMode（setFont）以及drawing Color，也就是说如果你传进来的字符串编码必须在font定义里面；

示例：

```
u8g2.setFont(u8g2_font_unifont_t_symbols);
u8g2.drawUTF8(5, 20, "Snowman: ☺");
```



3.2.2.16 u8g2.drawLine() —— 绘制竖直线

函数说明：

```
/**
 * 绘制竖直线
 * @param x 左上角坐标x
 * @param y 左上角坐标y
 * @param h 高度
 */
void U8G2::drawVLine(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t h)
```

3.2.2.17 u8g2.drawXBM()/drawXBMP() —— 绘制图像

函数说明：

```
/**
 * 绘制图像
 * @param x 左上角坐标x
 * @param y 左上角坐标y
 * @param w 图形宽度
 * @param h 图形高度
 * @param bitmap 图形内容
 * @Note 关联方法 setBitmapMode
 */
void U8G2::drawXBM(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w, u8g2_uint_t h, const uint8_t *bitmap)
void U8G2::drawXBMP(u8g2_uint_t x, u8g2_uint_t y, u8g2_uint_t w, u8g2_uint_t h, const uint8_t *bitmap)
```

注意点：

- drawXBM和drawXBMP区别在于 XBMP支持PROGMEM；

3.2.2.18 u8g2.firstPage()/nextPage() —— 绘制命令

函数说明：

```
/**
 * 绘制图像
 */
void U8G2::firstPage(void)
uint8_t U8G2::nextPage(void)
```

注意点：

- firstPage方法会把当前页码位置变成0；
- 修改内容处于firstPage和nextPage之间，每次都是重新渲染所有内容；

优势点：

- 该方法消耗的ram空间，比sendBuffer消耗的ram空间要少；

示例：

```
u8g2.firstPage();
do {
    /* all graphics commands have to appear within the loop body. */
    u8g2.setFont(u8g2_font_ncenB14_tr);
    u8g2.drawStr(0,20,"Hello World!");
} while ( u8g2.nextPage() );
```

库源码解析：

```
void u8g2_FirstPage(u8g2_t *u8g2)
{
    if ( u8g2->is_auto_page_clear )
    {
        //清除缓冲区
        u8g2_ClearBuffer(u8g2);
    }
    //设置当前缓冲区的Tile Row 一个Tile等于8个像素点的高度
    u8g2_SetBufferCurrTileRow(u8g2, 0);
}

uint8_t u8g2_NextPage(u8g2_t *u8g2)
{
    uint8_t row;
    u8g2_send_buffer(u8g2);
    row = u8g2->tile_curr_row;
    row += u8g2->tile_buf_height;
    if ( row >= u8g2_GetU8x8(u8g2)->display_info->tile_height )
    {
        //如果row已经到达最后一行，触发refreshDisplay调用，表示整个页面已经刷完了
        u8x8_RefreshDisplay( u8g2_GetU8x8(u8g2) );
        return 0;
    }
    if ( u8g2->is_auto_page_clear )
    {
```

```

//清除缓冲区
u8g2_ClearBuffer(u8g2);
}
//不断更新TileRow 这是非常关键的一步
u8g2_SetBufferCurrTileRow(u8g2, row);
return 1;
}

```

3.2.2.19 u8g2.print() —— 绘制内容

函数说明：

```

/**
 * 绘制内容
 * @Note 关联方法  setFont setCursor enableUTF8Print
 */
void U8G2::print(...)

```

示例：

```

u8g2.setFont(u8g2_font_ncenB14_tr);
u8g2.setCursor(0, 15);
u8g2.print("Hello World!");

```



3.2.2.20 u8g2.sendBuffer() —— 绘制缓冲区的内容

函数说明：

```

/**
 * 绘制缓冲区的内容
 * @Note 关联方法  clearBuffer
 */
void U8G2::sendBuffer(void)

```

注意点：

- sendBuffer的RAM占用空间大，需要结合构造器的buffer选项（请继续往下看，先有个概念）使用；
- 不管是firstPage、nextPage还是sendBuffer，都涉及到一个叫做 current page position的概念；

库源码解析：

```

void u8g2_SendBuffer(u8g2_t *u8g2)
{
    u8g2_send_buffer(u8g2);
    u8x8_RefreshDisplay( u8g2_GetU8x8(u8g2) );
}

static void u8g2_send_tile_row(u8g2_t *u8g2, uint8_t src_tile_row, uint8_t dest_tile_row)
{
    uint8_t *ptr;
    uint16_t offset;
    uint8_t w;

    w = u8g2_GetU8x8(u8g2)->display_info->tile_width;
    offset = src_tile_row;
    ptr = u8g2->tile_buf_ptr;
    offset *= w;
    offset *= 8;
    ptr += offset;
    u8x8_DrawTile(u8g2_GetU8x8(u8g2), 0, dest_tile_row, w, ptr);
}

/*
    write the buffer to the display RAM.
    For most displays, this will make the content visible to the user.
    Some displays (like the SSD1606) require a u8x8_RefreshDisplay()
*/
static void u8g2_send_buffer(u8g2_t *u8g2) U8X8_NOINLINE;
static void u8g2_send_buffer(u8g2_t *u8g2)
{
    uint8_t src_row;
    uint8_t src_max;
    uint8_t dest_row;
    uint8_t dest_max;

    src_row = 0;
    src_max = u8g2->tile_buf_height;
    dest_row = u8g2->tile_curr_row;
    dest_max = u8g2_GetU8x8(u8g2)->display_info->tile_height;

    do
    {
        u8g2_send_tile_row(u8g2, src_row, dest_row);
        src_row++;
        dest_row++;
    } while( src_row < src_max && dest_row < dest_max );
}

```

示例：

```

void loop(void) {
    u8g2.clearBuffer();
    // ... write something to the buffer
    u8g2.sendBuffer();
    delay(1000);
}

```

3.2.3 显示配置相关函数

3.2.3.1 u8g2.getAscent() —— 获取基准线以上的高度

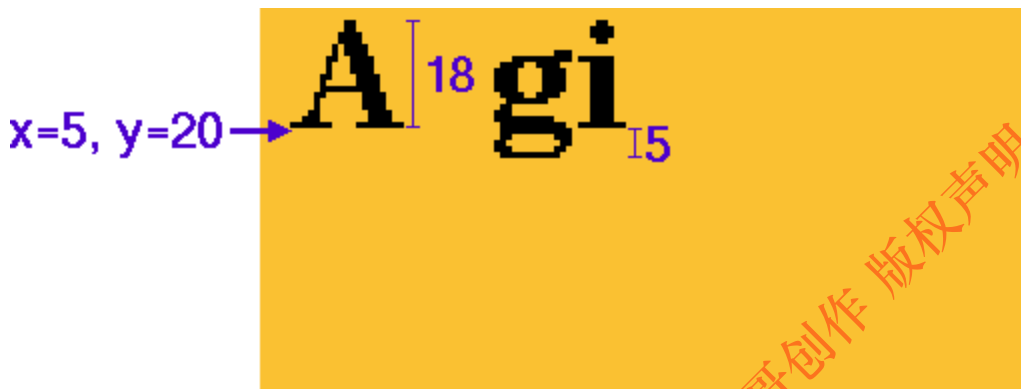
函数说明：

```
/**
 * 获取基准线以上的高度
 * @return 返回高度值
 * @Note 关联方法  setFont getDescent setFontRefHeightAll
 */
int8_t U8G2::getAscent(void)
```

注意点：

- 跟字体有关（setFont）；

示例：下面例子，ascent是18



3.2.3.2 u8g2.getDescent() —— 获取基准线以下的高度

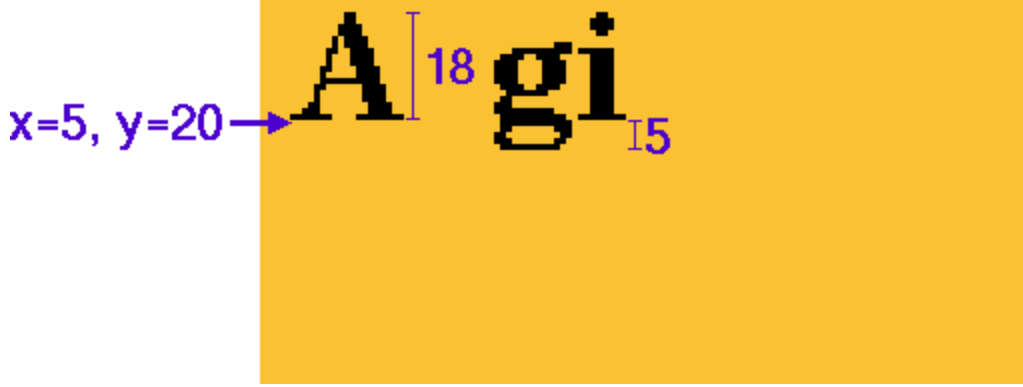
函数说明：

```
/**
 * 获取基准线以下的高度
 * @return 返回高度值
 * @Note 关联方法  setFont setFontRefHeightAll
 */
int8_t U8G2::getDescent(void)
```

注意点：

- 跟字体有关（setFont）；

示例：下面例子，descent是-5



3.2.3.3 u8g2.getDisplayHeight() —— 获取显示器的高度

函数说明：

```
/**
 * 获取显示器的高度
 * @return 返回高度值
 */
u8g2_uint_t getDisplayHeight(void)
```

3.2.3.4 u8g2.getDisplayWidth() —— 获取显示器的宽度

函数说明：

```
/**
 * 获取显示器的宽度
 * @return 返回宽度值
 */
u8g2_uint_t getDisplayWidth(void)
```

3.2.3.5 u8g2.getMaxCharHeight() —— 获取当前字体里的最大字符的高度

函数说明：

```
/**
 * 获取当前字体里的最大字符的高度
 * @return 返回高度值
 * @Note 关联方法 setFont
 */
u8g2_uint_t getMaxCharHeight(void)
```

注意点：

- 每一个字符在font字集中都是一个位图，位图有高度和宽度；

3.2.3.6 u8g2.getMaxCharWidth() —— 获取当前字体里的最大字符的宽度

函数说明：

```

/**
 * 获取当前字体里的最大字符的宽度
 * @return 返回宽度值
 * @Note 关联方法 setFont
 */
u8g2_uint_t getMaxCharWidth(void)

```

注意点：

- 每一个字符在font字集中都是一个位图，位图有高度和宽度；

3.2.3.7 u8g2.getStrWidth() —— 获取字符串的像素宽度

函数说明：

```

/**
 * 获取字符串的像素宽度
 * @param s 绘制字符串
 * @return 返回字符串的像素宽度值
 * @Note 关联方法 setFont drawStr
 */
u8g2_uint_t U8G2::getStrWidth(const char *s)

```

注意点：

- 像素宽度和当前font字体有关；

3.2.3.8 u8g2.getUTF8Width() —— 获取UTF-8字符串的像素宽度

函数说明：

```

/**
 * 获取UTF-8字符串的像素宽度
 * @param s 绘制字符串
 * @return 返回字符串的像素宽度值
 * @Note 关联方法 setFont drawStr
 */
u8g2_uint_t U8G2::getUTF8Width(const char *s)

```

注意点：

- 像素宽度和当前font字体有关；

3.2.3.9 u8g2.setAutoPageClear() —— 设置自动清除缓冲区

函数说明：

```

/**
 * 是否自动清除缓冲区
 * @param mode 0 表示关闭
 *             1 表示开启，默认是开启

```

```
*/  
void U8G2::setAutoPageClear(uint8_t mode)
```

注意点：

- 该方法用于 firstPage 和 nextPage（看上面的源码解析）；
- 建议该方法保持默认就好，如果用户禁止了，那么需要自己维护缓冲区的状态或者手动调用clearBuffer；

3.2.3.10 u8g2.setBitmapMode() —— 设置位图模式

函数说明：

```
/**  
 * 设置位图模式（定义drawXBM方法是否绘制背景颜色）  
 * @param is_transparent  
 *      0 绘制背景颜色，不透明，默认是该值  
 *      1 不绘制背景颜色，透明  
 * @Note 关联方法 drawXBM  
 */  
void U8G2::setBitmapMode(uint8_t is_transparent)
```

示例：

```
u8g2.setDrawColor(1);  
u8g2.setBitmapMode(0);  
u8g2.drawXBM(4,3, u8g2_logo_97x51_width, u8g2_logo_97x51_height, u8g2_logo_97x51_bits);  
u8g2.drawXBM(12,11, u8g2_logo_97x51_width, u8g2_logo_97x51_height, u8g2_logo_97x51_bits);
```



```
u8g2.setDrawColor(1);  
u8g2.setBitmapMode(1);  
u8g2.drawXBM(4,3, u8g2_logo_97x51_width, u8g2_logo_97x51_height, u8g2_logo_97x51_bits);  
u8g2.drawXBM(12,11, u8g2_logo_97x51_width, u8g2_logo_97x51_height, u8g2_logo_97x51_bits);
```



3.2.3.11 u8g2.setBusClock() —— 设置总线时钟

函数说明：

```
/**
 * 设置总线时钟(I2C SPI)
 * @param mode clock_speed 总线时钟频率(Hz)
 * @Note 关联方法 begin
 */
void U8G2::setBusClock(uint32_t clock_speed);
```

注意点：

- 仅仅Arduino平台支持；
- 必须在u8g2.begin() 或者 u8g2.initDisplay()之前调用；

3.2.3.12 u8g2.setClipWindow() —— 设置采集窗口大小

函数说明：

```
/**
 * 设置采集窗口，窗口范围从左上角(x0, y0)到右下角(x1, y1)
 * 也就是我们绘制的内容只能在规范范围内显示
 * @param x0 左上角x坐标
 * @param y0 左上角y坐标
 * @param x1 右上角x坐标
 * @param y1 右上角y坐标
 * @Note 关联方法 begin
 */
void U8G2::setClipWindow(u8g2_uint_t x0, u8g2_uint_t y0, u8g2_uint_t x1, u8g2_uint_t y1 );
```

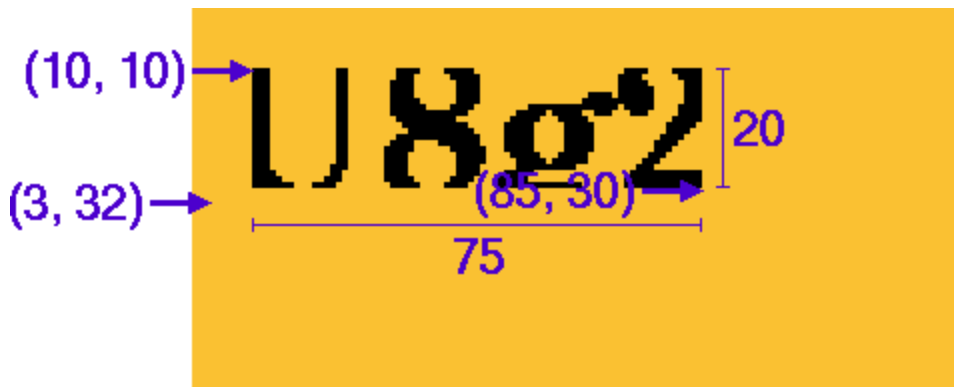
注意点：

- 可以通过 setMaxClipWindow 去掉该限制

```
void U8G2::setMaxClipWindow(void)
```

示例：

```
u8g2.setClipWindow(10, 10, 85, 30);
u8g2.setDrawColor(1);
u8g2.drawStr(3, 32, "U8g2");
```



3.2.3.13 u8g2.setCursor() —— 设置绘制光标位置

函数说明：

```
/**
 * 设置绘制光标位置(x,y)
 * @Note 关联方法 print
 */
void U8G2::setCursor(u8g2_uint_t x, u8g2_uint_t y)
```

示例：

```
u8g2.setFont(u8g2_font_ncenB14_tr);
u8g2.setCursor(0, 15);
u8g2.print("Hello World!");
```



3.2.3.14 u8g2.setDisplayRotation() —— 设置显示器的旋转角度

函数说明：

```
/**
 * 设置显示器的旋转角度
 * @param u8g2_cb 旋转选项
 *         U8G2_R0 不做旋转 水平
 *         U8G2_R1 旋转90度
```

```

*      U8G2_R2  旋转180度
*      U8G2_R3  旋转270度
*      U8G2_MIRROR  不做旋转 水平，显示内容是镜像的，暂时不理解
*/
void setDisplayRotation(const u8g2_cb_t *u8g2_cb)

```

3.2.3.15 u8g2.setDrawColor() —— 设置绘制颜色

函数说明：

```

/**
 * 设置绘制颜色（暂时还没有具体去了解用法）
 */
void U8G2::setDrawColor(uint8_t color)

```

3.2.3.16 u8g2.setFont() —— 设置字体集

这是一个非常重要的方法，非常重要！！！！

函数说明：

```

/**
 * 设置字体集（字体集用于字符串绘制方法或者glyph绘制方法）
 * @param font 具体的字体集
 * @Note 关联方法 drawUTF8 drawStr drawGlyph print
 */
void U8G2::setFont(const uint8_t *font)

```

Font会根据像素点高度做了很多区分，具体font请参考 [wiki](#)。如果我们需要用到中文字符，可以在wiki里面搜索一下chinese，你就会发现很多中文font，比如：

```

//支持UTF-8或者GB2312编码
u8g2_font_wqy15_t_chinese1
u8g2_font_wqy15_t_chinese2
u8g2_font_wqy15_t_chinese3
u8g2_font_wqy12_t_gb2312
u8g2_font_wqy12_t_gb2312a
.....

```

注意点：

- 中文字符集消耗内存大，请谨慎使用，可以用在Arduino 101等ram空间比较大的板上；

```

u8g2_font_wqy15_t_chinese1
BBX Width 16, Height 17, Capital A 11
Font Data Size: 12602 Bytes
32/0020  ! " # $ % & ' ( ) * + , - . /
48/0030  0 1 2 3 4 5 6 7 8 9 : ; < = > ?
64/0040  @ A B C D E F G H I J K L M N O
80/0050  P Q R S T U V W X Y Z [ \ ] ^ _
96/0060  ` a b c d e f g h i j k l m n o
112/0070 p q r s t u v w x y z { | } ~
128/0080  €
8208/2010  —

```

19968/4e00 一
20000/4e20
20016/4e30
20032/4e40 么
20048/4e50 也
20064/4e60
20096/4e80 书了 事些 于
20112/4e90 云互
20128/4ea0 交
20144/4eb0 人
20160/4ec0 从
20176/4ed0 他
20192/4ee0 以件 仪们 份
20208/4ef0
20240/4f10 优 会
20256/4f20 传
20288/4f40 但
20304/4f50 作
20320/4f60 你
20432/4fd0 保
20448/4fe0 信
20640/50a0 储
20672/50c0 像
20800/5140 充兆
20832/5160 入其 公
20848/5170 关 内 册
20864/5180
20976/51f0 出击
20992/5200 分切
21024/5220 删 制刷
21040/5230 到
21056/5240 前
21088/5260 剪
21152/52a0 务 千 动 博
21312/5340
21328/5350 单
21344/5360 卡
21360/5370 印
21424/53b0 去
21440/53c0 双
21456/53d0 发 取
21472/53e0 口 只 可
21488/53f0 台 号司
21504/5400 同 后
21536/5420 吧
21632/5480 和 响
21696/54c0 售
21792/5520
21824/5540 商 器
22112/5660
22224/56d0 回
22240/56e0 因
22256/56f0 在 国图
22304/5720
22320/5730 地 圾
22336/5740 址
22352/5750 坛
22400/5780 垃
22480/57d0 域
22672/5890 墙
22768/58f0 壳
22784/5900 备 复

单片机菜鸟博哥创作 版权声明

22800/5910
22816/5920
22832/5930
22896/5970
22912/5980
23376/5b50
23392/5b60
23440/5b90
23456/5ba0
23472/5bb0
23536/5bf0
23552/5c00
23600/5c30
23616/5c40
23632/5c50
24080/5e10
24096/5e20
24144/5e50
24176/5e70
24192/5e80
24208/5e90
24224/5ea0
24320/5f00
24336/5f10
24400/5f50
24464/5f90
24480/5fa0
24512/5fc0
24576/6000
24608/6020
24816/60f0
25088/6200
25104/6210

多
天
她
好
字
存
学
定
宝
实
家
对
宽
小
屏
属
帖
带
幕
年
幻
序
店
度
式
引
录
得
微
心
怀
性
想
戏
成我

头
如
子
客
室
就
局

至于用哪一个，看自己的需求了。

我们看看Font的命名规则：

```
<prefix> '_' <name> '_' <purpose> <char set>
```

其中：

- prefix基本上都是 u8g2；
- name 一般会挂钩上字符像素使用量，比如5X7
- purpose

<purpose>	描述
t	Transparent font, Do not use a background color.
h	All glyphs have common height(所有的图形有通用的高度).
m	All glyphs have common height and width (monospace).
8	All glyphs fit into a 8x8 pixel box.

- char set

<char set>	描述
f	The font includes up to 256 glyphs.
r	Only glyphs on the range of the ASCII codes 32 to 127 are included in the font.
u	Only glyphs on the range of the ASCII codes 32 to 95 (uppercase chars) are included in the font.
n	Only numbers and extra glyphs for writing date and time strings are included in the font.
...	Other custom character list.

注意点：

- U8G2库提供的font非常多，博主也暂时消化不了太多。如果我们使用中文的话，就去看看中文font就好；

示例：Fonts u8g2_font_5x7_tr and u8g2_font_pressstart2p_8u

```
u8g2_font_5x7_tr
BBX Width 5, Height 7, Capital A 6
Font Data Size: 841 Bytes
 32/0020  !"#$%&'()*+,-./
 48/0030  0123456789:;<=>?
 64/0040  @ABCDEFGHIJKLMNO
 80/0050  PQRSTUVWXYZ[\]^_
 96/0060  `abcdefghijklmnopqrstuvwxyz
112/0070  pqrstuvwxyz{|}~

The quick brown fox
jumps over the lazy dog.

u8g2_font_pressstart2p_8u
BBX Width 8, Height 7, Capital A 8
Font Data Size: 831 Bytes
 32/0020  !"#$%&'()*+,-./
 48/0030  0123456789:;<=>?
 64/0040  @ABCDEFGHIJKLMNO
 80/0050  PQRSTUVWXYZ[\]^_
```

3.2.3.17 u8g2.setFontDirection() —— 设置字体方向

函数说明：

```
/**
 * 定义字符串绘制或者图形绘制的方向
 * @param dir 方向
 * @param 关联方法 drawStr
 */
void U8G2::setFontDirection(uint8_t dir)
```

注意点：

- dir参数

Argument	String Rotation	Description
0	0 degree	Left to right
1	90 degree	Top to down
2	180 degree	Right to left

Argument	String Rotation	Description
3	270 degree	Down to top

示例：

```
u8g2.setFont(u8g2_font_ncenB14_tf);
u8g2.setFontDirection(0);
u8g2.drawStr(15, 20, "Abc");
u8g2.setFontDirection(1);
u8g2.drawStr(15, 20, "Abc");
```



3.2.4 缓存相关函数

缓存相关函数，一般不会去操作，了解即可；

3.2.4.1 u8g2.getBufferPtr() —— 获取缓存空间的地址

函数说明

```
/**
 * 获取缓存空间的地址
 * @return 返回缓存空间起始地址
 * @Note 关联方法 getBufferTileHeight, getBufferTileWidth, clearBuffer
 */
uint8_t *U8G2::getBufferPtr(void)
```

注意点：

- 缓存大小等于 8 * u8g2.getBufferTileHeight() * u8g2.getBufferTileWidth().

3.2.4.2 u8g2.getBufferTileHeight() —— 获取缓冲区的Tile高度

函数说明

```
/**
 * 获取缓冲区的Tile高度
 * @return 返回高度值
 */
uint8_t U8G2::getBufferTileHeight(void)
```

注意点：

- 一个tile等于8个像素点.

3.2.4.3 u8g2.getBufferTileWidth() —— 获取缓冲区的Tile宽度

函数说明

```
/**
 * 获取缓冲区的Tile宽度
 * @return 返回宽度值
 */
uint8_t U8G2::getBufferTileWidth(void)
```

注意点：

- 一个tile等于8个像素点.

3.2.4.4 u8g2.getBufferCurrTileRow() —— 获取缓冲区的当前Tile row

函数说明

```
/**
 * 获取缓冲区的当前Tile row行数
 * @return 返回当前的tilerow
 */
uint8_t U8G2::getBufferCurrTileRow(void)
```

注意点：

- 这个方法跟我们上面说到的page position相关.

3.2.4.5 u8g2.setBufferCurrTileRow() —— 设置缓冲区的当前Tile row

函数说明

```
/**
 * 设置缓冲区的当前Tile row
 * @param 当前的tilerow
 */
void U8G2::setBufferCurrTileRow(uint8_t row)
```

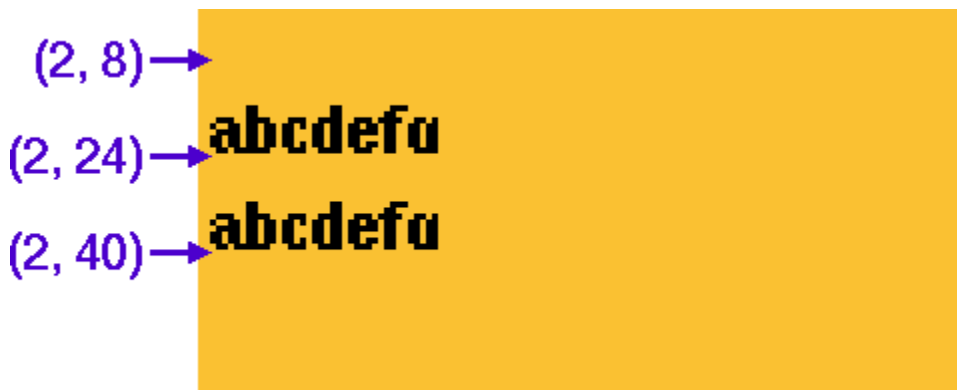
注意点：

- 在 firstPage/nextPage 循环时，由于底层调用了setBufferCurrTileRow，所以尽量不要自己手动调用该方法；

示例：

```
u8g2.setBufferCurrTileRow(0);           // let y=0 be the topmost row of the buffer
u8g2.clearBuffer();
u8g2.setFont(u8g2_font_helvb08_tr);
u8g2.drawStr(2, 8, "abcdefg");
```

```
u8g2.setBufferCurrTileRow(2);    // write the buffer to tile row 2 (y=16) on the display
u8g2.sendBuffer();
u8g2.setBufferCurrTileRow(4);    // write the same buffer to tile row 4 (y=32) on the display
u8g2.sendBuffer();
```



利用好该方法，我们可以实现部分更新；

4.如何运用U8G2库

前面博主介绍到U8G2适配了绝大部分的OLED，那么我们如何构建具体的OLED驱动呢？可分为以下几个顺序步骤：

- 区分显示器
- 选择物理总线方式
- 区分数字连线
- U8g2初始化
- U8g2绘制模式

4.1 区分显示器

首先，你需要知道OLED显示器的控制器型号以及屏幕大小。举个例子，博主手上有一块SSD1306 128X64的OLED，那么它的控制器就是SSD1306，屏幕大小是128X64。

其次，你所选择的OLED必须在U8g2库所支持的OLED列表中，具体可参考 [链接地址](#)。

4.2 选择物理总线方式

图像信息是通过物理总线方式发给OLED显示器。通常，我们的总线包括：

- 3SPI, 3-wire SPI：串行外围接口，依靠三个控制信号，Clock、Data、CS；
- 4SPI, 4-Wire SPI，跟3SPI一样，只是额外多了一条数据命令线，经常叫做D/C；
- I2C, IIC or TWI: SCL SDA；
- 8080:A 8-Bit bus which requires 8 data lines, chip select and a write strobe signal
- 6800: Another 8-Bit bus, but with a different protocol.

具体的OLED使用什么物理总线，我们需要查阅各自的数据手册。比如，博主的SSD1306就是IIC。

4.3 区分数字连线

知道了物理连线模式之后，我们一般都是把OLED连接到Arduino Board的输出引脚，也就是软件模拟具体总线协议。当然，如果有现成的物理总线端口那就更好了。

4.4 U8g2初始化

经历以上三步之后，我们就可以开始初始化出具体的OLED驱动了。比如，博主的IIC SSD1306 128X64 的OLED，就可以用以下初始化构造器（Builder设计模式，有空可以去了解一下）：

```
U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/  
U8X8_PIN_NONE); // All Boards without Reset of the Display
```

当然SSD1306还有其他构造器（具体可以参考 [wiki](#)）：

Controller "ssd1306", Display "128x64_noname"	Descirption
U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI(rotation, clock, data, cs, dc [, reset])	page buffer, size = 128 bytes
U8G2_SSD1306_128X64_NONAME_2_4W_SW_SPI(rotation, clock, data, cs, dc [, reset])	page buffer, size = 256 bytes
U8G2_SSD1306_128X64_NONAME_F_4W_SW_SPI(rotation, clock, data, cs, dc [, reset])	full framebuffer, size = 1024 bytes
U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI(rotation, cs, dc [, reset])	page buffer, size = 128 bytes

那么，我们这里就需要重点讲述一下构造器的规则。
构造器的名字包括以下几方面：

No	Description	Example
1	Prefix	U8G2
2	Display Controller	SSD1306
3	Display Name	128X64_NONAME
4	Buffer Size	1, 2 or F (full frame buffer)
5	Communication	4W_SW_SPI

它们之间使用"_"连接起来。其中：

- BufferSize，缓存大小

BufferSize	Description
1	保持一页的缓冲区，用于firstPage/nextPage的PageMode.
2	保持两页的缓冲区，用于firstPage/nextPage的PageMode..
F	获取整个屏幕的缓冲区，ram消耗大，一般用在ram空间比较大的arduino板子.

- Communication，通信协议

Communication	Description
4W_SW_SPI	4-wire (clock, data, cs and dc) software emulated SPI
4W_HW_SPI	4-wire (clock, data, cs and dc) hardware SPI (based on Arduino SPI library)
2ND_4W_HW_SPI	If supported, second 4-wire hardware SPI (based on Arduino SPI library)
3W_SW_SPI	3-wire (clock, data and cs) software emulated SPI
SW_I2C	Software emulated I2C/TWI
HW_I2C	Hardware I2C based on the Arduino Wire library
2ND_HW_I2C	If supported, use second hardware I2C (Arduino Wire lib)
6800	8-bit parallel interface, 6800 protocol
8080	8-bit parallel interface, 8080 protocol

- Rotation （软件模拟总线前提下的构造器的第一个参数）

Rotation/Mirror	Description
U8G2_R0	No rotation, landscape
U8G2_R1	90 degree clockwise rotation
U8G2_R2	180 degree clockwise rotation
U8G2_R3	270 degree clockwise rotation
U8G2_MIRROR	No rotation, landscape, display content is mirrored (v2.6.x)

所以，一个完整的例子为：

```
#include <Arduino.h>
#include <U8g2lib.h>
#include <SPI.h>
#include <Wire.h>

U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, 13, 11, 10, 8);

void setup(void) {
  u8g2.begin();
}

void loop(void) {
  u8g2.firstPage();
  do {
    u8g2.setFont(u8g2_font_ncenB14_tr);
    u8g2.drawStr(0,24,"Hello World!");
  } while ( u8g2.nextPage() );
}
```

那么，我们来看看到底构造器里面做了什么操作？还是以博主使用的SSD1306 128X64为例子：

```
/**
 * SSD1306 4线软件模拟SPI
```

```
*/  
U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI(.....)
```

类定义：

```
/**  
 * SSD1306构造器，继承U8G2  
 */  
class U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI : public U8G2 {  
    public: U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI(const u8g2_cb_t *rotation, uint8_t clock, uint8_t data,  
    uint8_t cs, uint8_t dc, uint8_t reset = U8X8_PIN_NONE) : U8G2() {  
        //配置SSD1306  
        u8g2_Setup_ssd1306_128x64_noname_1(&u8g2, rotation, u8x8_byte_arduino_4wire_sw_spi,  
        u8x8_gpio_and_delay_arduino);  
        //设置通信协议  
        u8x8_SetPin_4Wire_SW_SPI(getU8x8(), clock, data, cs, dc, reset);  
    }  
};
```

从上面代码看出，默认调用了父类U8G2的构造函数，我们看看它里面做了什么：

```
class U8G2 : public Print  
{  
    protected:  
        u8g2_t u8g2;  
        u8x8_char_cb cpp_next_cb; /* the cpp interface has its own decoding function for the Arduino print  
command */  
    public:  
        u8g2_uint_t tx, ty;  
  
        U8G2(void) {  
            //设置Arduino print函数的解码方法，这里是ASCII，当然也有UTF-8  
            cpp_next_cb = u8x8_ascii_next;  
            //屏幕初始化  
            home();  
        }  
        .....  
};
```

- U8G2类构造函数主要是定义好解码方法以及初始化屏幕（包括重置原点）；

然后我们看看，u8g2_Setup_ssd1306_128x64_noname_1做了什么：

```
void u8g2_Setup_ssd1306_128x64_noname_1(u8g2_t *u8g2, const u8g2_cb_t *rotation, u8x8_msg_cb byte_cb,  
u8x8_msg_cb gpio_and_delay_cb)  
{  
    uint8_t tile_buf_height;  
    //定义好缓存空间 记住 这里是 1 page mode  
    uint8_t *buf;  
    //配置屏幕  
    u8g2_SetupDisplay(u8g2, u8x8_d_ssd1306_128x64_noname, u8x8_cad_001, byte_cb, gpio_and_delay_cb);  
    //生成buf 这里是128 bytes  
    buf = u8g2_m_16_8_1(&tile_buf_height);  
    //初始化buf  
    u8g2_SetupBuffer(u8g2, buf, tile_buf_height, u8g2_ll_hvline_vertical_top_lsb, rotation);  
}
```

```

/*=====*/
/*
   This procedure is called after setting up the display (u8x8 structure).
   --> This is the central init procedure for u8g2 object
*/
void u8g2_SetupBuffer(u8g2_t *u8g2, uint8_t *buf, uint8_t tile_buf_height, u8g2_draw_ll_hvline_cb
ll_hvline_cb, const u8g2_cb_t *u8g2_cb)
{
    u8g2->font = NULL;
    //u8g2->kerneling = NULL;
    //u8g2->get_kerneling_cb = u8g2_GetNullKerneling;

    //u8g2->ll_hvline = u8g2_ll_hvline_vertical_top_lsb;
    u8g2->ll_hvline = ll_hvline_cb;

    u8g2->tile_buf_ptr = buf;
    u8g2->tile_buf_height = tile_buf_height;

    u8g2->tile_curr_row = 0; //页码 这是一个很重要的参数

    u8g2->font_decode.is_transparent = 0; /* issue 443 */
    u8g2->bitmap_transparency = 0;

    u8g2->draw_color = 1;
    u8g2->is_auto_page_clear = 1; //自动清除

    u8g2->cb = u8g2_cb;
    u8g2->cb->update_dimension(u8g2);
#ifdef U8G2_WITH_CLIP_WINDOW_SUPPORT
    u8g2_SetMaxClipWindow(u8g2); /* assign a clip window and call the update() procedure */
#else
    u8g2->cb->update_page_win(u8g2);
#endif

    u8g2_SetFontPosBaseline(u8g2); /* issue 195 */

#ifdef U8G2_WITH_FONT_ROTATION
    u8g2->font_decode.dir = 0;
#endif
}

```

可以总结几点：

- 配置初始化屏幕
- 配置初始化缓冲区

注意点：

- 上面讲解涉及到了绘制模式，请参考下一节；

4.5 U8g2绘制模式

U8g2支持三种绘制模式：

- Full screen buffer mode，全屏缓存模式
- Page mode (This is the U8glib picture loop) 分页模式

- U8x8, character only mode 仅仅支持普通字符

4.5.1 Full screen buffer mode

特点：

- 绘制速度快
- 所有的绘制方法都可以使用
- 需要大量的ram空间

构造器：

- 构造器必须带有F，比如：

```
U8G2_ST7920_128X64_F_SW_SPI(rotation, clock, data, cs [, reset])
```

用法：

1. 清除缓冲区 u8g2.clearBuffer()
2. 操作一些绘制方法
3. 发送缓冲区的内容到显示器 u8g2.sendBuffer().

示例代码：

```
void setup(void) {  
    u8g2.begin();  
}  
  
void loop(void) {  
    u8g2.clearBuffer();  
    u8g2.setFont(u8g2_font_ncenB14_tr);  
    u8g2.drawStr(0, 20, "Hello World!");  
    u8g2.sendBuffer();  
}
```

4.5.2 Page mode

特点：

- 绘制速度慢
- 所有的绘制方法都可以使用
- 需要少量的ram空间

构造器：

- 构造器必须带有“1”或者2，比如：

```
U8G2_ST7920_128X64_1_SW_SPI(rotation, clock, data, cs [, reset])
```

用法：

1. 调用 u8g2.firstPage()
2. 开始一个 do while 循环
3. 在循环内部 操作一些绘制方法
4. 不断判断 u8g2.nextPage()

示例代码：

```
void setup(void) {
    u8g2.begin();
}

void loop(void) {
    u8g2.firstPage();
    do {
        u8g2.setFont(u8g2_font_ncenB14_tr);
        u8g2.drawStr(0,24,"Hello World!");
    } while ( u8g2.nextPage() );
}
```

4.5.3 U8x8 character mode

特点：

- 绘制速度快
- 并不是对所有的显示器都有效
- 图形绘制不可用
- 不需要ram空间

构造器：

- 使用U8X8构造器，比如：

```
U8X8_ST7565_EA_D0GM128_4W_SW_SPI(clock, data, cs, dc [, reset])
```

用法：

- 所有绘制命令是直接把数据写到显示器

示例代码：

```
void setup(void) {
    u8x8.begin();
}

void loop(void) {
    u8x8.setFont(u8x8_font_chroma48medium8_r);
    u8x8.drawString(0,1,"Hello World!");
}
```

5.U8g2库具体用例

上面讲解了那么多理论知识，是时候去实际运用。博主在这里选用了Arduino Mega2560 以及 NodeMcu ESP8266开发板，结合手上的SSD1306 OLED去讲解例子。

5.1 SSD1306 + Mega2560

5.1.1 Hello World

实验内容：

- Full screen buffer mode
- 自带字体显示“Hello World”
- I2C总线

实验器材：

- Mega2560 + SSD1306 128X64

实验代码：

```
/*
   HelloWorld.ino
*/

#include <Arduino.h>
#include <U8g2lib.h>

#ifdef U8X8_HAVE_HW_SPI
#include <SPI.h>
#else
#ifdef U8X8_HAVE_HW_I2C
#include <Wire.h>
#endif
#endif

/*
   U8g2lib Example Overview:
   Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because
of RAM consumption
   Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.
   U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8
Text only.
*/

// Please UNCOMMENT one of the constructor lines below
// U8g2 Constructor List (Frame Buffer)
// The complete list is available here: https://github.com/olikraus/u8g2/wiki/u8g2setupcpp
// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not
connected
//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing
//U8G2_SSD1306_128X64_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy
(Production, Kickstarter Edition)
//U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
reset=*/ 8);
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_128X64_ALT0_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME
```

```
variant, but may solve the "every 2nd line skipped" problem
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // All Boards without Reset of the Display
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C
//U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/
17); // ESP32 Thing, HW I2C with pin remapping
//U8G2_SSD1306_128X64_NONAME_F_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_VCOMH0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but maximizes setContrast() range
//U8G2_SSD1306_128X64_ALT0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but may solve the "every 2nd line skipped" problem
//U8G2_SH1106_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1106_128X64_VCOMH0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but maximizes setContrast() range
//U8G2_SH1106_128X64_WINSTAR_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but uses updated SH1106 init sequence
//U8G2_SH1106_72X40_WISE_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_64X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_PIMORONI_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_SEEED_128X128_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_96X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1108_160X160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1122_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1306_128X32_UNIVISION_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit
ESP8266/32u4/ARM Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_128X32_WINSTAR_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_64X48_ER_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED
breakout board, Uno: A4=SDA, A5=SCL, 5V powered
//U8G2_SSD1306_48X64_WINSTAR_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_1F_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_96X16_ER_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED
//U8G2_SSD1322_NHD_256X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1322_NHD_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD0323_0S128064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
```

```
//U8G2_SSD0323_OS128064_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,  
/* reset=*/ 8); // experimental driver for ER-OLED018-1  
//U8G2_SSD1327_SEEED_96X96_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/  
U8X8_PIN_NONE); // Seeedstudio Grove OLED 96x96  
//U8G2_SSD1327_SEEED_96X96_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seeedstudio Grove OLED  
96x96  
//U8G2_SSD1327_EA_W128128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8);  
//U8G2_SSD1327_EA_W128128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1327_EA_W128128_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1327_EA_W128128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1327_MIDAS_128X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1327_MIDAS_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1327_WS_128X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8);  
//U8G2_SSD1327_WS_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1329_128X96_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1329_128X96_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_ADAFRUIT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_ADAFRUIT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X64_ADAFRUIT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X64_ADAFRUIT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1309_128X64_NONAME0_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1309_128X64_NONAME0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1309_128X64_NONAME2_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1309_128X64_NONAME2_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1317_96X96_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*  
reset=*/ 8); // not tested, not confirmed  
//U8G2_SSD1317_96X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,  
not confirmed  
//U8G2_SSD1318_128X96_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*  
reset=*/ 8);  
//U8G2_SSD1318_128X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_LD7032_60X32_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*  
reset=*/ 8); // SW SPI Nano Board  
//U8G2_LD7032_60X32_F_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);  
// NOT TESTED!  
//U8G2_UC1701_EA_D0GS102_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,  
/* reset=*/ 8);  
//U8G2_UC1701_EA_D0GS102_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_PCD8544_84X48_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*  
reset=*/ 8); // Nokia 5110 Display  
//U8G2_PCD8544_84X48_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia 5110  
Display  
//U8G2_PCF8812_96X65_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*  
reset=*/ 8); // Could be also PCF8814  
//U8G2_PCF8812_96X65_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could  
be also PCF8814  
//U8G2_HX1230_96X68_F_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);  
//U8G2_HX1230_96X68_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*  
reset=*/ 8);  
//U8G2_KS0108_128X64_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
```

```

/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=clock=14, RW=data=13, RS=CS
//U8G2_ST7920_128X64_F_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_F_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=clock=14, RW=data=13, RS=CS
//U8G2_ST7565_EA_DOGM128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_DOGM128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_64128N_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_EA_DOGM132_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_DOGM132_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7565_ZOLEN_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_LM6059_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_LX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_ERC12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_ALT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // contrast improved version for ERC12864
//U8G2_ST7565_ERC12864_ALT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12832_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_JLX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_JLX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_PI_132X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9,
/* reset=*/ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7567_JLX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);

```

```
//U8G2_ST7567_ENH_DG128064I_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_64X32_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX172104_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX19296_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX19296_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX256128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 9, /* data=*/ 8, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX240160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX25664_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, enable U8g2 16 bit mode for this display
//U8G2_NT7534_TG12864R_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_NT7534_TG12864R_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7588_JLX12864_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ 5);
//U8G2_ST7588_JLX12864_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 5);
//U8G2_IST3020_ERC19264_F_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 28, /*cs=*/ 32, /*dc=*/ 30, /*reset=*/ 31); // Connect WR pin with GND
//U8G2_IST3020_ERC19264_F_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 29, /*cs=*/ 32, /*dc=*/ 30, /*reset=*/ 31); // Connect RD pin with 3.3V
//U8G2_IST3020_ERC19264_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LC7981_160X80_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_160X160_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X64_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RW with GND
//U8G2_SED1520_122X32_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc=*/ A0, /*e1=*/ A3, /*e2=*/ A2, /*reset=*/ A4); // Set R/W to low!
//U8G2_T6963_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14, /*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
```

```

/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RD with +5V, FG with GND
//U8G2_SED1330_240X128_F_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10, /*dc=*/
9, /*reset=*/ 8); // A0 is dc pin!
//U8G2_RA8835_NHD_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1604_JLX19264_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1604_JLX19264_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_UC1608_ERC24064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1608_240X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1610_EA_DOGXL160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Due, SW SPI, DOGM240 Test Board
//U8G2_UC1611_EA_DOGXL240_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Due, SW SPI, DOGXL240 Test Board
//U8G2_UC1611_EW50850_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
//U8G2_UC1638_160X128_F_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!
//U8G2_MAX7219_64X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_MAX7219_32X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_MAX7219_8X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_LS013B7DH03_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_F_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

```

// End of constructor list

```

void setup(void) {
    u8g2.begin();
}

void loop(void) {
    u8g2.clearBuffer();           // 清除内部缓冲区
    u8g2.setFont(u8g2_font_ncenB08_tr); // choose a suitable font
    u8g2.drawStr(0,10,"Hello World!"); // write something to the internal memory
    u8g2.drawStr(0,20,"This is bro cainiao!"); // write something to the internal memory
    u8g2.drawStr(0,30,"Welcome to U8G2!"); // write something to the internal memory
    u8g2.sendBuffer();           // transfer internal memory to the display
    delay(1000);
}

```

实验结果：



5.1.2 PrintUTF8 打印中文字符

实验内容：

- Full screen buffer mode
- 自带字体显示“我是单片机菜博”
- I2C总线

实验器材：

- Mega2560 + SSD1306 128X64

实验代码：

```

/*
    PrintUTF8.ino
*/

#include <Arduino.h>
#include <U8g2lib.h>

#ifdef U8X8_HAVE_HW_SPI
#include <SPI.h>

```

```

#endif
#ifdef U8X8_HAVE_HW_I2C
#include <Wire.h>
#endif

/*
U8g2lib Example Overview:
Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because
of RAM consumption
Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.
U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8
Text only.

*/

// Please UNCOMMENT one of the constructor lines below
// U8g2 Constructor List (Frame Buffer)
// The complete list is available here: https://github.com/olikraus/u8g2/wiki/u8g2setupcpp
// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not
// connected
//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing
//U8G2_SSD1306_128X64_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy
(Production, Kickstarter Edition)
//U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
reset=*/ 8);
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_128X64_ALTO_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME
variant, but may solve the "every 2nd line skipped" problem
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // All Boards without Reset of the Display
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C
//U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/
17); // ESP32 Thing, HW I2C with pin remapping
//U8G2_SSD1306_128X64_NONAME_F_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_VCOMH0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but maximizes setContrast() range
//U8G2_SSD1306_128X64_ALTO_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but may solve the "every 2nd line skipped" problem
//U8G2_SH1106_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1106_128X64_VCOMH0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but maximizes setContrast() range
//U8G2_SH1106_128X64_WINSTAR_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but uses updated SH1106 init sequence
//U8G2_SH1106_72X40_WISE_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_64X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_PIMORONI_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_SEEED_128X128_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_96X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

```



```
//U8G2_SH1122_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SH1122_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable  
U8G2_16BIT in u8g2.h  
//U8G2_SSD1306_128X32_UNIVISION_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/  
U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED  
//U8G2_SSD1306_128X32_UNIVISION_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/  
U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED  
//U8G2_SSD1306_128X32_UNIVISION_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit  
ESP8266/32u4/ARM Boards + FeatherWing OLED  
//U8G2_SSD1306_128X32_UNIVISION_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*  
data=*/ SDA); // pin remapping with ESP8266 HW I2C  
//U8G2_SSD1306_128X32_WINSTAR_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*  
data=*/ SDA); // pin remapping with ESP8266 HW I2C  
//U8G2_SSD1306_64X48_ER_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED  
breakout board, Uno: A4=SDA, A5=SCL, 5V powered  
//U8G2_SSD1306_48X64_WINSTAR_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1306_64X32_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1306_64X32_1F_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1306_96X16_ER_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED  
//U8G2_SSD1322_NHD_256X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h  
//U8G2_SSD1322_NHD_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable  
U8G2_16BIT in u8g2.h  
//U8G2_SSD1322_NHD_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8);  
//U8G2_SSD1322_NHD_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1325_NHD_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8);  
//U8G2_SSD1325_NHD_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD0323_OS128064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,  
/* reset=*/ 8);  
//U8G2_SSD0323_OS128064_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,  
/* reset=*/ 8); // experimental driver for ER-OLED018-1  
//U8G2_SSD1327_SEEED_96X96_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/  
U8X8_PIN_NONE); // Seeedstudio Grove OLED 96x96  
//U8G2_SSD1327_SEEED_96X96_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seeedstudio Grove OLED  
96x96  
//U8G2_SSD1327_EA_W128128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8);  
//U8G2_SSD1327_EA_W128128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1327_EA_W128128_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1327_EA_W128128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1327_MIDAS_128X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1327_MIDAS_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1327_WS_128X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/  
9, /* reset=*/ 8);  
//U8G2_SSD1327_WS_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1329_128X96_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1329_128X96_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_ADAFRUIT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X32_ADAFRUIT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X64_ADAFRUIT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1305_128X64_ADAFRUIT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1309_128X64_NONAME0_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
```

```
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1317_96X96_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // not tested, not confirmed
//U8G2_SSD1317_96X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,
not confirmed
//U8G2_SSD1318_128X96_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_SSD1318_128X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LD7032_60X32_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*
reset=*/ 8); // SW SPI Nano Board
//U8G2_LD7032_60X32_F_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);
// NOT TESTED!
//U8G2_UC1701_EA_D0GS102_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1701_EA_D0GS102_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_PCD8544_84X48_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Nokia 5110 Display
//U8G2_PCD8544_84X48_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia 5110
Display
//U8G2_PCF8812_96X65_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Could be also PCF8814
//U8G2_PCF8812_96X65_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could
be also PCF8814
//U8G2_HX1230_96X68_F_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_HX1230_96X68_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_KS0108_128X64_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */ , /* data=*/ 16 /* A2 */ , /* CS=*/ 17
/* A3 */ , /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */ , /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */ , /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */ , /* data=*/ 16 /* A2 */ , /* CS=*/ 17
/* A3 */ , /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=clock=14, RW=data=13, RS=CS
//U8G2_ST7920_128X64_F_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_F_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=clock=14, RW=data=13, RS=CS
//U8G2_ST7565_EA_D0GM128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_D0GM128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_64128N_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_EA_D0GM132_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_D0GM132_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7565_ZOLEN_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_LM6059_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
```

```

reset= */ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9, /*
reset= */ 8);
//U8G2_ST7565_LX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_ERC12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_ERC12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_ERC12864_ALT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8); // contrast improved version for ERC12864
//U8G2_ST7565_ERC12864_ALT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_NHD_C12832_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_NHD_C12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_NHD_C12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_JLX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_JLX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_PI_132X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 7, /* dc= */ 9,
/* reset= */ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 7, /* dc= */ 9, /* reset= */ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 7, /* dc= */ 9, /*
reset= */ 8);
//U8G2_ST7567_JLX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 7, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064I_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064I_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_64X32_F_HW_I2C u8g2(U8G2_R0, /* reset= */ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST75256_JLX172104_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST75256_JLX19296_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST75256_JLX19296_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST75256_JLX256128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Enable
U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Enable
U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 9, /* data= */ 8, /* cs= */ 7, /* dc= */ 6, /*
reset= */ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 7, /* dc= */ 6, /* reset= */ 5); // MKR Zero,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Enable
U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST75256_JLX240160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST75256_JLX25664_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset= */ 8); // Due, 2nd I2C, enable U8g2 16 bit

```

```
mode for this display
//U8G2_NT7534_TG12864R_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_NT7534_TG12864R_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7588_JLX12864_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ 5);
//U8G2_ST7588_JLX12864_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 5);
//U8G2_I2T3020_ERC19264_F_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 28, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect WR pin with GND
//U8G2_I2T3020_ERC19264_F_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 29, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect RD pin with 3.3V
//U8G2_I2T3020_ERC19264_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_LC7981_160X80_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_160X160_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X64_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_SED1520_122X32_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc=*/ A0, /*e1=*/ A3, /*e2=*/ A2, /*
reset=*/ A4); // Set R/W to low!
//U8G2_T6963_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RD with +5V, FG with GND
//U8G2_SED1330_240X128_F_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10, /*dc=*/
9, /*reset=*/ 8); // A0 is dc pin!
//U8G2_RA8835_NHD_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1604_JLX19264_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1604_JLX19264_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_UC1608_ERC24064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1608_240X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1610_EA_DOGXL160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Due, SW SPI, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Due, SW SPI, DOGXL240 Test Board
//U8G2_UC1611_EW50850_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
```

```

//U8G2_UC1638_160X128_F_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!
//U8G2_MAX7219_64X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
//U8G2_MAX7219_32X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
//U8G2_MAX7219_8X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
//U8G2_LS013B7DH03_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_F_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

// End of constructor list

void setup(void) {
    u8g2.begin();
    u8g2.enableUTF8Print(); // enable UTF8 support for the Arduino print() function
}

void loop(void) {
    // u8g2.setFont(u8g2_font_unifont_t_chinese2); // use chinese2 for all the glyphs of "我是单片机菜鸟"
    u8g2.setFont(u8g2_font_wqy15_t_chinese2); // use chinese2 for all the glyphs of "我是单片机菜鸟"
    u8g2.setFontDirection(0);
    u8g2.clearBuffer();
    u8g2.setCursor(0, 15);
    u8g2.print("你好");
    u8g2.setCursor(0, 40);
    u8g2.print("我是单片机菜博"); // Chinese "Hello World"
    u8g2.sendBuffer();

    delay(1000);
}

```

实验结果：



5.1.3 U8G2 logo

实验内容：

- Full screen buffer mode
- 绘制logo
- I2C总线

实验器材：

- Mega2560 + SSD1306 128X64

实验代码：

单片机菜鸟博哥创作 版权声明

```
/*
  U8g2Logo.ino
*/

#include <Arduino.h>
#include <U8g2lib.h>

#ifdef U8X8_HAVE_HW_SPI
#include <SPI.h>
#else
#ifdef U8X8_HAVE_HW_I2C
#include <Wire.h>
#endif
#endif

/*
  U8g2lib Example Overview:
  Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because
  of RAM consumption
  Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.
  U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8
  Text only.

  This is a frame buffer example.
*/
```

```

*/
// Please UNCOMMENT one of the constructor lines below
// U8g2 Constructor List (Frame Buffer)
// The complete list is available here: https://github.com/olikraus/u8g2/wiki/u8g2setupcpp
// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not
// connected
//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing
//U8G2_SSD1306_128X64_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy
(Production, Kickstarter Edition)
//U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
reset=*/ 8);
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_128X64_ALTO_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME
variant, but may solve the "every 2nd line skipped" problem
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // All Boards without Reset of the Display
//U8G2_SSD1306_128X64_NONAME_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C
//U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/
17); // ESP32 Thing, HW I2C with pin remapping
//U8G2_SSD1306_128X64_NONAME_F_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_F_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_VCOMH0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but maximizes setContrast() range
//U8G2_SSD1306_128X64_ALTO_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but may solve the "every 2nd line skipped" problem
//U8G2_SH1106_128X64_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1106_128X64_VCOMH0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but maximizes setContrast() range
//U8G2_SH1106_128X64_WINSTAR_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but uses updated SH1106 init sequence
//U8G2_SH1106_72X40_WISE_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_64X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_PIMORONI_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_SEEED_128X128_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_128X128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_96X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1108_160X160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1122_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1306_128X32_UNIVISION_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit
ESP8266/32u4/ARM Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_128X32_WINSTAR_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_64X48_ER_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED

```

```
board, Uno: A4=SDA, A5=SCL, 5V powered
//U8G2_SSD1306_48X64_WINSTAR_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_1F_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_96X16_ER_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED
//U8G2_SSD1322_NHD_256X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_256X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1322_NHD_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD0323_OS128064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_SSD0323_OS128064_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // experimental driver for ER-OLED018-1
//U8G2_SSD1327_SEEED_96X96_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Seeedstudio Grove OLED 96x96
//U8G2_SSD1327_SEEED_96X96_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seeedstudio Grove OLED
96x96
//U8G2_SSD1327_EA_W128128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_EA_W128128_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_MIDAS_128X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_MIDAS_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1317_96X96_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // not tested, not confirmed
//U8G2_SSD1317_96X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,
not confirmed
//U8G2_SSD1318_128X96_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_SSD1318_128X96_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LD7032_60X32_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*
reset=*/ 8); // SW SPI Nano Board
```

```

//U8G2_LD7032_60X32_F_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);
// NOT TESTED!
//U8G2_UC1701_EA_D0GS102_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1701_EA_D0GS102_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_PCD8544_84X48_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Nokia 5110 Display
//U8G2_PCD8544_84X48_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia 5110
Display
//U8G2_PCF8812_96X65_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Could be also PCF8814
//U8G2_PCF8812_96X65_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could
be also PCF8814
//U8G2_HX1230_96X68_F_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_HX1230_96X68_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_KS0108_128X64_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_F_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=14, RW=13, RS=CS
//U8G2_ST7920_128X64_F_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_F_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=14, RW=13, RS=CS
//U8G2_ST7565_EA_D0GM128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_D0GM128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_64128N_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_EA_D0GM132_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_D0GM132_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7565_ZOLEN_128X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_LM6059_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_LX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_ERC12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_ALT_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // contrast improved version for ERC12864
//U8G2_ST7565_ERC12864_ALT_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,

```

```

/* reset=*/ 8);
//U8G2_ST7565_NHD_C12832_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_JLX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_JLX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_PI_132X64_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9,
/* reset=*/ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7567_JLX12864_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_64X32_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX172104_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX19296_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX19296_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX256128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 9, /* data=*/ 8, /* cs=*/ 7, /* dc=*/ 6, /*
reset=*/ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX240160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX25664_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, enable U8g2 16 bit
mode for this display
//U8G2_NT7534_TG12864R_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_NT7534_TG12864R_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7588_JLX12864_F_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ 5);
//U8G2_ST7588_JLX12864_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ 5);
//U8G2_IST3020_ERC19264_F_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 28, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect WR pin with GND
//U8G2_IST3020_ERC19264_F_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 29, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect RD pin with 3.3V
//U8G2_IST3020_ERC19264_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_LC7981_160X80_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND

```

```

//U8G2_LC7981_160X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X64_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_SED1520_122X32_F u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc=*/ A0, /*e1=*/ A3, /*e2=*/ A2, /*
reset=*/ A4); // Set R/W to low!
//U8G2_T6963_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RD with +5V, FG with GND
//U8G2_SED1330_240X128_F_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10, /*dc=*/
9, /*reset=*/ 8); // A0 is dc pin!
//U8G2_RA8835_NHD_240X128_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_F_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1604_JLX19264_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1604_JLX19264_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_UC1608_ERC24064_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1608_240X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1610_EA_DOGXL160_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Due, SW SPI, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_F_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Due, SW SPI, DOGXL240 Test Board
//U8G2_UC1611_EW50850_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_F_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
//U8G2_UC1638_160X128_F_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!

```

```

//U8G2_MAX7219_64X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
//U8G2_MAX7219_32X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
//U8G2_MAX7219_8X8_F_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 10, /* dc=*/
U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE);
//U8G2_LS013B7DH03_128X128_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_F_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_F_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable*/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

// End of constructor list

// #define MINI_LOGO

void setup(void) {
  pinMode(9, OUTPUT);
  digitalWrite(9, 0); // default output in I2C mode for the SSD1306 test shield: set the i2c adr to 0

  u8g2.begin();
}

void drawLogo(void)
{
  u8g2.setFontMode(1); // Transparent
#ifdef MINI_LOGO

  u8g2.setFontDirection(0);
  u8g2.setFont(u8g2_font_inb16_mf);
  u8g2.drawStr(0, 22, "U");

  u8g2.setFontDirection(1);
  u8g2.setFont(u8g2_font_inb19_mn);
  u8g2.drawStr(14, 8, "8");

  u8g2.setFontDirection(0);
  u8g2.setFont(u8g2_font_inb16_mf);
  u8g2.drawStr(36, 22, "g");
  u8g2.drawStr(48, 22, "\xb2");

  u8g2.drawHLine(2, 25, 34);
  u8g2.drawHLine(3, 26, 34);
  u8g2.drawVLine(32, 22, 12);
  u8g2.drawVLine(33, 23, 12);
#else

  u8g2.setFontDirection(0);
  u8g2.setFont(u8g2_font_inb24_mf);
  u8g2.drawStr(0, 30, "U");

  u8g2.setFontDirection(1);
  u8g2.setFont(u8g2_font_inb30_mn);
  u8g2.drawStr(21, 8, "8");

  u8g2.setFontDirection(0);
  u8g2.setFont(u8g2_font_inb24_mf);
  u8g2.drawStr(51, 30, "g");

```

```

u8g2.drawStr(67, 30, "\xb2");

u8g2.drawLine(2, 35, 47);
u8g2.drawLine(3, 36, 47);
u8g2.drawLine(45, 32, 12);
u8g2.drawLine(46, 33, 12);

#endif
}

void drawURL(void)
{
#ifdef MINI_LOGO
    u8g2.setFont(u8g2_font_4x6_tr);
    if ( u8g2.getDisplayHeight() < 59 )
    {
        u8g2.drawStr(89, 20, "github.com");
        u8g2.drawStr(73, 29, "/olikraus/u8g2");
    }
    else
    {
        u8g2.drawStr(1, 54, "github.com/olikraus/u8g2");
    }
}
#endif
}

void loop(void) {
    u8g2.clearBuffer();
    drawLogo();
    drawURL();
    u8g2.sendBuffer();
    delay(1000);
}

```

实验结果：



5.1.4 Weather

实验内容：

- Full screen buffer mode
- 绘制天气符号以及温度 滚动显示内容
- I2C总线

实验器材：

- Mega2560 + SSD1306 128X64

实验代码：

```
/*  
  
    Weather.ino  
  
*/  
  
#include <Arduino.h>  
#include <U8g2lib.h>  
  
#ifdef U8X8_HAVE_HW_SPI  
#include <SPI.h>  
#endif  
#ifdef U8X8_HAVE_HW_I2C  
#include <Wire.h>  
#endif  
  
/*  
    U8g2lib Example Overview:  
    Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because  
    of RAM consumption  
    Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.  
    U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8  
    Text only.  
  
    This is a page buffer example.  
*/  
  
// Please UNCOMMENT one of the constructor lines below  
// U8g2 Constructor List (Picture Loop Page Buffer)  
// The complete list is available here: https://github.com/olikraus/u8g2/wiki/u8g2setupcpp  
// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not  
// connected  
//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing  
//U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy  
(Production, Kickstarter Edition)  
//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);  
//U8G2_SSD1306_128X64_NONAME_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*  
reset=*/ 8);  
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);  
//U8G2_SSD1306_128X64_ALT0_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME  
variant, but may solve the "every 2nd line skipped" problem  
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);  
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/  
U8X8_PIN_NONE); // All Boards without Reset of the Display
```

```
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C
//U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/
17); // ESP32 Thing, HW I2C with pin remapping
//U8G2_SSD1306_128X64_NONAME_1_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but maximizes setContrast() range
//U8G2_SSD1306_128X64_ALTO_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but may solve the "every 2nd line skipped" problem
//U8G2_SH1106_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1106_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but maximizes setContrast() range
//U8G2_SH1106_128X64_WINSTAR_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but uses updated SH1106 init sequence
//U8G2_SH1106_72X40_WISE_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_64X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_PIMORONI_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_SEEED_128X128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1108_160X160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1122_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit
ESP8266/32u4/ARM Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_128X32_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_64X48_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED
breakout board, Uno: A4=SDA, A5=SCL, 5V powered
//U8G2_SSD1306_48X64_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_1F_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_96X16_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED
//U8G2_SSD1322_NHD_256X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1322_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // experimental driver for ER-OLED018-1
//U8G2_SSD1327_SEEED_96X96_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
```

```

U8X8_PIN_NONE); // Seedstudio Grove OLED 96x96
//U8G2_SSD1327_SEED_96X96_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seedstudio Grove OLED
96x96
//U8G2_SSD1327_EA_W128128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_EA_W128128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_MIDAS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_MIDAS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1317_96X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // not tested, not confirmed
//U8G2_SSD1317_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,
not confirmed
//U8G2_SSD1318_128X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_SSD1318_128X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LD7032_60X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*
reset=*/ 8); // SW SPI Nano Board
//U8G2_LD7032_60X32_1_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);
// NOT TESTED!
//U8G2_UC1701_EA_D0GS102_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1701_EA_D0GS102_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_PCD8544_84X48_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Nokia 5110 Display
//U8G2_PCD8544_84X48_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia
5110 Display
//U8G2_PCF8812_96X65_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Could be also PCF8814
//U8G2_PCF8812_96X65_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could
be also PCF8814
//U8G2_HX1230_96X68_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_HX1230_96X68_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_KS0108_128X64_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,

```

```
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=clock=14, RW=data=13, RS=CS
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=clock=14, RW=data=13, RS=CS
//U8G2_ST7565_EA_DOGM128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_DOGM128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_EA_DOGM132_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_DOGM132_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7565_ZOLEN_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_LM6059_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_LX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_ERC12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_ALT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // contrast improved version for ERC12864
//U8G2_ST7565_ERC12864_ALT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12832_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_PI_132X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9,
/* reset=*/ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7567_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
```

```
//U8G2_ST7567_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX172104_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX19296_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX19296_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 9, /* data=*/ 8, /* cs=*/ 7, /* dc=*/ 6, /*
reset=*/ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX240160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX25664_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, enable U8g2 16 bit
mode for this display
//U8G2_NT7534_TG12864R_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_NT7534_TG12864R_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7588_JLX12864_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ 5);
//U8G2_ST7588_JLX12864_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 5);
//U8G2_IST3020_ERC19264_1_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 28, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect WR pin with GND
//U8G2_IST3020_ERC19264_1_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 29, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect RD pin with 3.3V
//U8G2_IST3020_ERC19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_LC7981_160X80_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_160X160_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X64_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_SED1520_122X32_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc=*/ A0, /*e1=*/ A3, /*e2=*/ A2, /*
reset=*/ A4); // Set R/W to low!
//U8G2_T6963_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_SED1330_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
```

```

15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_RA8835_NHD_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1601_128X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_UC1601_128X32_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_UC1604_JLX19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1604_JLX19264_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_UC1608_ERC24064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1608_240X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1610_EA_DOGXL160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EW50850_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
//U8G2_UC1638_160X128_1_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!
//U8G2_LS013B7DH03_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

```

```
// End of constructor list
```

```

#define SUN 0
#define SUN_CLOUD 1
#define CLOUD 2
#define RAIN 3
#define THUNDER 4

```

```
void drawWeatherSymbol(u8g2_uint_t x, u8g2_uint_t y, uint8_t symbol)
```

```

{
    // fonts used:
    // u8g2_font_open_iconic_embedded_6x_t
    // u8g2_font_open_iconic_weather_6x_t
    // encoding values, see: https://github.com/olikraus/u8g2/wiki/fntgrpiconic

    switch(symbol)
    {
        case SUN:
            u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);
            u8g2.drawGlyph(x, y, 69);
            break;
        case SUN_CLOUD:
            u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);
            u8g2.drawGlyph(x, y, 65);
            break;
        case CLOUD:
            u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);
            u8g2.drawGlyph(x, y, 64);
            break;
        case RAIN:
            u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);
            u8g2.drawGlyph(x, y, 67);
            break;
        case THUNDER:
            u8g2.setFont(u8g2_font_open_iconic_embedded_6x_t);
            u8g2.drawGlyph(x, y, 67);
            break;
    }
}

void drawWeather(uint8_t symbol, int degree)
{
    //绘制天气符号
    drawWeatherSymbol(0, 48, symbol);
    //绘制温度
    u8g2.setFont(u8g2_font_logisoso32_tf);
    u8g2.setCursor(48+3, 42);
    u8g2.print(degree);
    u8g2.print("C"); // requires enableUTF8Print()
}

/*
    Draw a string with specified pixel offset.
    The offset can be negative.
    Limitation: The monochrome font with 8 pixel per glyph
*/
void drawScrollString(int16_t offset, const char *s)
{
    static char buf[36]; // should for screen with up to 256 pixel width
    size_t len;
    size_t char_offset = 0;
    u8g2_uint_t dx = 0;
    size_t visible = 0;

    u8g2.setDrawColor(0); // clear the scrolling area
    u8g2.drawBox(0, 49, u8g2.getDisplayWidth()-1, u8g2.getDisplayHeight()-1);
    u8g2.setDrawColor(1); // set the color for the text

```

```

len = strlen(s);
if ( offset < 0 )
{
    char_offset = (-offset)/8;
    dx = offset + char_offset*8;
    if ( char_offset >= u8g2.getDisplayWidth()/8 )
        return;
    visible = u8g2.getDisplayWidth()/8-char_offset+1;
    strncpy(buf, s, visible);
    buf[visible] = '\0';
    u8g2.setFont(u8g2_font_8x13_mf);
    u8g2.drawStr(char_offset*8-dx, 62, buf);
}
else
{
    char_offset = offset / 8;
    if ( char_offset >= len )
        return; // nothing visible
    dx = offset - char_offset*8;
    visible = len - char_offset;
    if ( visible > u8g2.getDisplayWidth()/8+1 )
        visible = u8g2.getDisplayWidth()/8+1;
    strncpy(buf, s+char_offset, visible);
    buf[visible] = '\0';
    u8g2.setFont(u8g2_font_8x13_mf);
    u8g2.drawStr(-dx, 62, buf);
}
}

void draw(const char *s, uint8_t symbol, int degree)
{
    int16_t offset = -(int16_t)u8g2.getDisplayWidth();
    int16_t len = strlen(s);

    u8g2.clearBuffer(); // clear the internal memory
    drawWeather(symbol, degree); // draw the icon and degree only once
    for(;;) // then do the scrolling
    {
        drawScrollString(offset, s); // no clearBuffer required, screen will be partially cleared here
        u8g2.sendBuffer(); // transfer internal memory to the display

        delay(20);
        offset+=2;
        if ( offset > len*8+1 )
            break;
    }
}

void setup(void) {

    /* U8g2 Project: SSD1306 Test Board */
    pinMode(10, OUTPUT);
    pinMode(9, OUTPUT);

    digitalWrite(10, 0);
    digitalWrite(9, 0);

    /* U8g2 Project: T6963 Test Board */
    //pinMode(18, OUTPUT);

```

```
//digitalWrite(18, 1);

/* U8g2 Project: KS0108 Test Board */
//pinMode(16, OUTPUT);
//digitalWrite(16, 0);

/* U8g2 Project: LC7981 Test Board, connect RW to GND */
//pinMode(17, OUTPUT);
//digitalWrite(17, 0);

/* U8g2 Project: Pax Instruments Shield: Enable Backlight */
//pinMode(6, OUTPUT);
//digitalWrite(6, 0);

u8g2.begin();
u8g2.enableUTF8Print();
}

void loop(void) {
  draw("What a beautiful day!", SUN, 27);
  draw("The sun's come out!", SUN_CLOUD, 19);
  draw("It's raining cats and dogs.", RAIN, 8);
  draw("That sounds like thunder.", THUNDER, 12);
  draw("It's stopped raining", CLOUD, 15);
}
```

实验结果：





5.1.5 Page Mode显示中文

实验内容：

- Page Mode
- 显示中文
- I2C总线

实验器材：

- Mega2560 + SSD1306 128X64

实验代码：

单片机菜鸟博哥创作 版权声明

```
/*
  Chinese.ino
*/

#include <Arduino.h>
#include <U8g2lib.h>

#ifdef U8X8_HAVE_HW_SPI
#include <SPI.h>
#endif
#ifdef U8X8_HAVE_HW_I2C
#include <Wire.h>
#endif

/*
  U8g2lib Example Overview:
  Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because
  of RAM consumption
  Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.
  U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8

```

Text only.

This is a page buffer example.

*/

// Please UNCOMMENT one of the constructor lines below

// U8g2 Constructor List (Picture Loop Page Buffer)

// The complete list is available here: <https://github.com/olikraus/u8g2/wiki/u8g2setupcpp>

// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not connected

//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing

//U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy (Production, Kickstarter Edition)

//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SSD1306_128X64_NONAME_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);

U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);

//U8G2_SSD1306_128X64_ALTO_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME variant, but may solve the "every 2nd line skipped" problem

//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);

//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ U8X8_PIN_NONE); // All Boards without Reset of the Display

//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/ U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C

//U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/ 17); // ESP32 Thing, HW I2C with pin remapping

//U8G2_SSD1306_128X64_NONAME_1_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10, /*dc=*/ 9, /*reset=*/ 8);

//U8G2_SSD1306_128X64_NONAME_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10, /*dc=*/ 9, /*reset=*/ 8);

//U8G2_SSD1306_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as the NONAME variant, but maximizes setContrast() range

//U8G2_SSD1306_128X64_ALTO_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as the NONAME variant, but may solve the "every 2nd line skipped" problem

//U8G2_SH1106_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);

//U8G2_SH1106_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as the NONAME variant, but maximizes setContrast() range

//U8G2_SH1106_128X64_WINSTAR_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as the NONAME variant, but uses updated SH1106 init sequence

//U8G2_SH1106_72X40_WISE_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SH1107_64X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SH1107_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SH1107_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);

//U8G2_SH1107_PIMORONI_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);

//U8G2_SH1107_SEEED_128X128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ U8X8_PIN_NONE);

//U8G2_SH1107_SEEED_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);

//U8G2_SH1107_SEEED_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SH1108_160X160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);

//U8G2_SH1122_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h

//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/ U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED

//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED

//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit ESP8266/32u4/ARM Boards + FeatherWing OLED

//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /* data=*/ SDA); // pin remapping with ESP8266 HW I2C

```
//U8G2_SSD1306_128X32_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_64X48_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED
breakout board, Uno: A4=SDA, A5=SCL, 5V powered
//U8G2_SSD1306_48X64_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_1F_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_96X16_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED
//U8G2_SSD1322_NHD_256X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1322_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // experimental driver for ER-OLED018-1
//U8G2_SSD1327_SEEED_96X96_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Seeedstudio Grove OLED 96x96
//U8G2_SSD1327_SEEED_96X96_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seeedstudio Grove OLED
96x96
//U8G2_SSD1327_EA_W128128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_EA_W128128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_MIDAS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_MIDAS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1317_96X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // not tested, not confirmed
//U8G2_SSD1317_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,
not confirmed
//U8G2_SSD1318_128X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
```

```

//U8G2_SSD1318_128X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LD7032_60X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*
reset=*/ 8); // SW SPI Nano Board
//U8G2_LD7032_60X32_1_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);
// NOT TESTED!
//U8G2_UC1701_EA_D0GS102_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1701_EA_D0GS102_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_PCD8544_84X48_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Nokia 5110 Display
//U8G2_PCD8544_84X48_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia
5110 Display
//U8G2_PCF8812_96X65_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Could be also PCF8814
//U8G2_PCF8812_96X65_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could
be also PCF8814
//U8G2_HX1230_96X68_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_HX1230_96X68_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_KS0108_128X64_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=clock=14, RW=data=13, RS=CS
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=clock=14, RW=data=13, RS=CS
//U8G2_ST7565_EA_D0GM128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_D0GM128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_EA_D0GM132_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_D0GM132_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7565_ZOLEN_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_LM6059_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_LX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_ERC12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_ALT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // contrast improved version for ERC12864

```

```
//U8G2_ST7565_ERC12864_AL_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12832_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_PI_132X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9,
/* reset=*/ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7567_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_64X32_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX172104_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX19296_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX19296_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 9, /* data=*/ 8, /* cs=*/ 7, /* dc=*/ 6, /*
reset=*/ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX240160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX25664_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, enable U8g2 16 bit
mode for this display
//U8G2_NT7534_TG12864R_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_NT7534_TG12864R_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7588_JLX12864_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ 5);
//U8G2_ST7588_JLX12864_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 5);
//U8G2_IST3020_ERC19264_1_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 28, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect WR pin with GND
//U8G2_IST3020_ERC19264_1_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 29, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect RD pin with 3.3V
//U8G2_IST3020_ERC19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
```

```

/* reset= */ 8);
//U8G2_LC7981_160X80_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_160X160_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X64_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_SED1520_122X32_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc=*/ A0, /*e1=*/ A3, /*e2=*/ A2, /*
reset=*/ A4); // Set R/W to low!
//U8G2_T6963_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_SED1330_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_RA8835_NHD_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1601_128X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_UC1601_128X32_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_UC1604_JLX19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1604_JLX19264_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_UC1608_ERC24064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1608_240X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1610_EA_DOGXL160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EW50850_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
//U8G2_UC1638_160X128_1_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/

```

```

9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!
//U8G2_LS013B7DH03_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

// End of constructor list

void setup(void) {
    u8g2.begin();
    u8g2.enableUTF8Print(); // enable UTF8 support for the Arduino print() function
}

void loop(void) {
    u8g2.setFont(u8g2_font_unifont_t_chinese2); // use chinese2 for all the glyphs of "你好世界"
    //u8g2.setFont(u8g2_font_b10_t_japanese1); // all the glyphs of "こんにちは世界" are already included in
japanese1: Larning Level 1-6
    u8g2.setFontDirection(0);
    u8g2.firstPage();
    do {
        u8g2.setCursor(0, 15);
        u8g2.print("Hello World!");
        u8g2.setCursor(0, 40);
        u8g2.print("你好世界"); // Chinese "Hello World"
    } while ( u8g2.nextPage() );
    delay(1000);
}

```

5.2 SSD1306 + NodeMcu

5.2.1 Page Mode显示中文

实验内容：

- Page Mode
- 显示中文
- I2C总线（NodeMcu的I2C引脚请参考博主ESP8266博文 [ESP8266开发之旅 基础篇③ ESP8266与Arduino的开发说明](#)）

实验器材：

- NodeMcu + SSD1306 128X64

实验代码：

```

/*
    Chinese.ino
*/

#include <Arduino.h>
#include <U8g2lib.h>

#ifdef U8X8_HAVE_HW_SPI
#include <SPI.h>
#endif
#ifdef U8X8_HAVE_HW_I2C
#include <Wire.h>
#endif

/*
    U8g2lib Example Overview:
    Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because
of RAM consumption
    Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.
    U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8
Text only.

    This is a page buffer example.
*/

// Please UNCOMMENT one of the constructor lines below
// U8g2 Constructor List (Picture Loop Page Buffer)
// The complete list is available here: https://github.com/olikraus/u8g2/wiki/u8g2setupcpp
// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not
connected
//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing
//U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy
(Production, Kickstarter Edition)
//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
reset=*/ 8);
U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_128X64_ALT0_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME
variant, but may solve the "every 2nd line skipped" problem
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // All Boards without Reset of the Display
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C
//U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/
17); // ESP32 Thing, HW I2C with pin remapping
//U8G2_SSD1306_128X64_NONAME_1_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but maximizes setContrast() range
//U8G2_SSD1306_128X64_ALT0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but may solve the "every 2nd line skipped" problem
//U8G2_SH1106_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1106_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but maximizes setContrast() range
//U8G2_SH1106_128X64_WINSTAR_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same

```

```
as the NAME variant, but uses updated SH1106 init sequence
//U8G2_SH1106_72X40_WISE_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_64X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_PIMORONI_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_SEEED_128X128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1108_160X160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1122_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit
ESP8266/32u4/ARM Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_128X32_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_64X48_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED
breakout board, Uno: A4=SDA, A5=SCL, 5V powered
//U8G2_SSD1306_48X64_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_1F_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_96X16_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED
//U8G2_SSD1322_NHD_256X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1322_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // experimental driver for ER-OLED018-1
//U8G2_SSD1327_SEEED_96X96_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Seeedstudio Grove OLED 96x96
//U8G2_SSD1327_SEEED_96X96_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seeedstudio Grove OLED
96x96
//U8G2_SSD1327_EA_W128128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_EA_W128128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_MIDAS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_MIDAS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
```

```
//U8G2 SSD1305_128X32_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1305_128X32_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1305_128X32_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1305_128X32_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1305_128X64_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1305_128X64_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1309_128X64_NONAME0_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1309_128X64_NONAME0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1309_128X64_NONAME2_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1309_128X64_NONAME2_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2 SSD1317_96X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // not tested, not confirmed
//U8G2 SSD1317_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,
not confirmed
//U8G2 SSD1318_128X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2 SSD1318_128X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LD7032_60X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*
reset=*/ 8); // SW SPI Nano Board
//U8G2_LD7032_60X32_1_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);
// NOT TESTED!
//U8G2_UC1701_EA_DOGS102_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 15, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1701_EA_DOGS102_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_PCD8544_84X48_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Nokia 5110 Display
//U8G2_PCD8544_84X48_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia
5110 Display
//U8G2_PCF8812_96X65_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Could be also PCF8814
//U8G2_PCF8812_96X65_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could
be also PCF8814
//U8G2_HX1230_96X68_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_HX1230_96X68_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_KS0108_128X64_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=clock=14, RW=data=13, RS=CS
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=clock=14, RW=data=13, RS=CS
//U8G2_ST7565_EA_DOGM128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_DOGM128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
```

```

reset= */ 8);
//U8G2_ST7565_64128N_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_EA_DOGM132_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_DOGM132_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8);
//U8G2_ST7565_ZOLEN_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_LM6059_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9, /*
reset= */ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9, /*
reset= */ 8);
//U8G2_ST7565_LX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_ERC12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_ERC12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_ERC12864_ALT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8); // contrast improved version for ERC12864
//U8G2_ST7565_ERC12864_ALT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_NHD_C12832_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_NHD_C12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_NHD_C12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7565_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST7565_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_PI_132X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 7, /* dc= */ 9,
/* reset= */ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 7, /* dc= */ 9, /* reset= */ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 7, /* dc= */ 9, /*
reset= */ 8);
//U8G2_ST7567_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 7, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7567_64X32_1_HW_I2C u8g2(U8G2_R0, /* reset= */ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST75256_JLX172104_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST75256_JLX19296_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_ST75256_JLX19296_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 9, /* data= */ 8, /* cs= */ 7, /* dc= */ 6, /*
reset= */ 5); // MKR Zero, Enable U8g2 16 bit mode for this display

```

```
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX240160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX25664_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, enable U8g2 16 bit
mode for this display
//U8G2_NT7534_TG12864R_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_NT7534_TG12864R_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7588_JLX12864_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/ 5);
//U8G2_ST7588_JLX12864_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 5);
//U8G2_IST3020_ERC19264_1_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 28, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect WR pin with GND
//U8G2_IST3020_ERC19264_1_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable=*/ 29, /*cs=*/ 32,
/*dc=*/ 30, /*reset=*/ 31); // Connect RD pin with 3.3V
//U8G2_IST3020_ERC19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_LC7981_160X80_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_160X160_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_LC7981_240X64_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect RW with GND
//U8G2_SED1520_122X32_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc=*/ A0, /*e1=*/ A3, /*e2=*/ A2, /*
reset=*/ A4); // Set R/W to low!
//U8G2_T6963_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr=*/ 17, /*cs/ce=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_SED1330_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14, /*dc=*/
15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_RA8835_NHD_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 17, /*cs=*/ 14,
/*dc=*/ 15, /*reset=*/ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1601_128X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_UC1601_128X32_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_UC1604_JLX19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1604_JLX19264_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_UC1608_ERC24064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1608_240X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // SW SPI, Due ERC24064-1 Test Setup
```

```

//U8G2_UC1610_EA_DOGXL160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8); // Due, 2nd I2C, DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EW50850_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
//U8G2_UC1638_160X128_1_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!
//U8G2_LS013B7DH03_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

// End of constructor list

void setup(void) {
    u8g2.begin();
    u8g2.enableUTF8Print(); // enable UTF8 support for the Arduino print() function
}

void loop(void) {
    u8g2.setFont(u8g2_font_unifont_t_chinese2); // use chinese2 for all the glyphs of "你好世界"
    //u8g2.setFont(u8g2_font_b10_t_japanese1); // all the glyphs of "こんにちは世界" are already included in
japanese1: Lerner Level 1-6
    u8g2.setFontDirection(0);
    u8g2.firstPage();
    do {
        u8g2.setCursor(0, 15);
        u8g2.print("Hello World!");
        u8g2.setCursor(0, 40);
        u8g2.print("你好世界"); // Chinese "Hello World"
    } while ( u8g2.nextPage() );
    delay(1000);
}

```

代码和上面Mega2560一样，测试效果一样，这里不重复贴测试图；

5.2.2 Weather

实验内容：

- Full screen buffer mode
- 绘制天气符号以及温度 滚动显示内容
- I2C总线 (NodeMcu的I2C引脚请参考博主ESP8266博文 [ESP8266开发之旅 基础篇③ ESP8266与Arduino的开发说明](#))

实验器材：

- NodeMcu + SSD1306 128X64

实验代码：

```
/*  
  
Weather.ino  
  
Demonstrates:  
- Weather icons from open-iconic set.  
- Text scrolling.  
- Partial screen update  
  
This example requires display height >= 64.  
  
Universal 8bit Graphics Library (https://github.com/olikraus/u8g2/)  
  
Copyright (c) 2016, olikraus@gmail.com  
All rights reserved.  
  
Redistribution and use in source and binary forms, with or without modification,  
are permitted provided that the following conditions are met:  
  
* Redistributions of source code must retain the above copyright notice, this list  
of conditions and the following disclaimer.  
  
* Redistributions in binary form must reproduce the above copyright notice, this  
list of conditions and the following disclaimer in the documentation and/or other  
materials provided with the distribution.  
  
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND  
CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES,  
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF  
MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE  
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR  
CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,  
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT  
NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;  
LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER  
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,  
STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)  
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF  
ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.  
  
*/  
  
#include <Arduino.h>  
#include <U8g2lib.h>
```

```

#ifndef U8X8_HAVE_HW_SPI
#include <SPI.h>
#endif
#ifdef U8X8_HAVE_HW_I2C
#include <Wire.h>
#endif

/*
  U8g2lib Example Overview:
  Frame Buffer Examples: clearBuffer/sendBuffer. Fast, but may not work with all Arduino boards because
  of RAM consumption
  Page Buffer Examples: firstPage/nextPage. Less RAM usage, should work with all Arduino boards.
  U8x8 Text Only Example: No RAM usage, direct communication with display controller. No graphics, 8x8
  Text only.

  This is a page buffer example.
*/

// Please UNCOMMENT one of the constructor lines below
// U8g2 Constructor List (Picture Loop Page Buffer)
// The complete list is available here: https://github.com/olikraus/u8g2/wiki/u8g2setupcpp
// Please update the pin numbers according to your setup. Use U8X8_PIN_NONE if the reset pin is not
// connected
//U8G2_NULL u8g2(U8G2_R0); // null device, a 8x8 pixel display which does nothing
//U8G2_SSD1306_128X64_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 12, /* dc=*/ 4, /* reset=*/ 6); // Arduboy
(Production, Kickstarter Edition)
//U8G2_SSD1306_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
reset=*/ 8);
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_128X64_ALTO_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // same as the NONAME
variant, but may solve the "every 2nd line skipped" problem
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // All Boards without Reset of the Display
//U8G2_SSD1306_128X64_NONAME_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 16, /* data=*/ 17, /* reset=*/
U8X8_PIN_NONE); // ESP32 Thing, pure SW emulated I2C
//U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ 16, /* data=*/
17); // ESP32 Thing, HW I2C with pin remapping
//U8G2_SSD1306_128X64_NONAME_1_6800 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_NONAME_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8);
//U8G2_SSD1306_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but maximizes setContrast() range
//U8G2_SSD1306_128X64_ALTO_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same as
the NONAME variant, but may solve the "every 2nd line skipped" problem
//U8G2_SH1106_128X64_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1106_128X64_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1106_128X64_VCOMH0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but maximizes setContrast() range
//U8G2_SH1106_128X64_WINSTAR_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // same
as the NONAME variant, but uses updated SH1106 init sequence
//U8G2_SH1106_72X40_WISE_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_64X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1107_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_PIMORONI_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ 8);
//U8G2_SH1107_SEEED_128X128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/

```

```
U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_128X128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SH1107_SEEED_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1108_160X160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SH1122_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 21, /* data=*/ 20, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather M0 Basic Proto + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Adafruit Feather ESP8266/32u4 Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Adafruit
ESP8266/32u4/ARM Boards + FeatherWing OLED
//U8G2_SSD1306_128X32_UNIVISION_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_128X32_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE, /* clock=*/ SCL, /*
data=*/ SDA); // pin remapping with ESP8266 HW I2C
//U8G2_SSD1306_64X48_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.66" OLED
breakout board, Uno: A4=SDA, A5=SCL, 5V powered
//U8G2_SSD1306_48X64_WINSTAR_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_NONAME_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_64X32_1F_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1306_96X16_ER_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // EastRising 0.69" OLED
//U8G2_SSD1322_NHD_256X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Enable U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_256X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Enable
U8G2_16BIT in u8g2.h
//U8G2_SSD1322_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1322_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1325_NHD_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_SSD0323_OS128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1326_ER_256X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // experimental driver for ER-OLED018-1
//U8G2_SSD1327_SEEED_96X96_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ SCL, /* data=*/ SDA, /* reset=*/
U8X8_PIN_NONE); // Seeedstudio Grove OLED 96x96
//U8G2_SSD1327_SEEED_96X96_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE); // Seeedstudio Grove OLED
96x96
//U8G2_SSD1327_EA_W128128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_EA_W128128_1_SW_I2C u8g2(U8G2_R0, /* clock=*/ 5, /* data=*/ 4, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_EA_W128128_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_SSD1327_MIDAS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_MIDAS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_SSD1327_WS_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1329_128X96_NONAME_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_NONAME_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X32_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
```

```
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1305_128X64_ADAFRUIT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME0_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /*
dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1309_128X64_NONAME2_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_SSD1317_96X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // not tested, not confirmed
//U8G2_SSD1317_96X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // not tested,
not confirmed
//U8G2_SSD1318_128X96_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_SSD1318_128X96_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_LD7032_60X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* cs=*/ 9, /* dc=*/ 10, /*
reset=*/ 8); // SW SPI Nano Board
//U8G2_LD7032_60X32_1_4W_SW_I2C u8g2(U8G2_R0, /* clock=*/ 11, /* data=*/ 12, /* reset=*/ U8X8_PIN_NONE);
// NOT TESTED!
//U8G2_UC1701_EA_DOGS102_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_UC1701_EA_DOGS102_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_PCD8544_84X48_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Nokia 5110 Display
//U8G2_PCD8544_84X48_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Nokia
5110 Display
//U8G2_PCF8812_96X65_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Could be also PCF8814
//U8G2_PCF8812_96X65_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Could
be also PCF8814
//U8G2_HX1230_96X68_1_3W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* reset=*/ 8);
//U8G2_HX1230_96X68_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_KS0108_128X64_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ U8X8_PIN_NONE, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_KS0108_ERM19264_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*dc=*/ 17, /*cs0=*/ 14,
/*cs1=*/ 15, /*cs2=*/ 16, /* reset=*/ U8X8_PIN_NONE); // Set R/W to low!
//U8G2_ST7920_192X32_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ U8X8_PIN_NONE,
/*dc=*/ 17, /*reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_192X32_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18 /* A4 */, /*cs=*/
U8X8_PIN_NONE, /*dc/rs=*/ 17 /* A3 */, /*reset=*/ 15 /* A1 */); // Remember to set R/W to 0
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 18 /* A4 */, /* data=*/ 16 /* A2 */, /* CS=*/ 17
/* A3 */, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_SW_SPI u8g2(U8G2_R0, /* clock=*/ 14, /* data=*/ 13, /* CS=*/ 15, /* reset=*/ 16);
// Feather HUZZAH ESP8266, E=clock=14, RW=data=13, RS=CS
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 10, /* reset=*/ 8);
//U8G2_ST7920_128X64_1_HW_SPI u8g2(U8G2_R0, /* CS=*/ 15, /* reset=*/ 16); // Feather HUZZAH ESP8266,
E=clock=14, RW=data=13, RS=CS
//U8G2_ST7565_EA_DOGM128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_EA_DOGM128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_64128N_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_EA_DOGM132_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ U8X8_PIN_NONE); // DOGM132 Shield
//U8G2_ST7565_EA_DOGM132_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ U8X8_PIN_NONE); //
DOGM132 Shield
//U8G2_ST7565_ZOLEN_128X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
```

```
9, /* reset=*/ 8);
//U8G2_ST7565_ZOLEN_128X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_LM6059_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // Adafruit ST7565 GLCD
//U8G2_ST7565_LM6059_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Adafruit
ST7565 GLCD
//U8G2_ST7565_LX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7565_LX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_ERC12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_ERC12864_ALT_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // contrast improved version for ERC12864
//U8G2_ST7565_ERC12864_ALT_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // contrast
improved version for ERC12864
//U8G2_ST7565_NHD_C12832_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12832_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_NHD_C12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7565_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST7565_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_PI_132X64_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9,
/* reset=*/ 8); // Pax Instruments Shield, LCD_BL=6
//U8G2_ST7567_PI_132X64_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8); // Pax
Instruments Shield, LCD_BL=6
//U8G2_ST7567_JLX12864_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 7, /* dc=*/ 9, /*
reset=*/ 8);
//U8G2_ST7567_JLX12864_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8);
//U8G2_ST7567_ENH_DG128064I_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST7567_64X32_1_HW_I2C u8g2(U8G2_R0, /* reset=*/ U8X8_PIN_NONE);
//U8G2_ST75256_JLX172104_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX172104_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX19296_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8);
//U8G2_ST75256_JLX19296_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8);
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_W0256X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 9, /* data=*/ 8, /* cs=*/ 7, /* dc=*/ 6, /*
reset=*/ 5); // MKR Zero, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 7, /* dc=*/ 6, /* reset=*/ 5); // MKR Zero,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // Uno, Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX256160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ 9, /* reset=*/ 8); // Uno,
Enable U8g2 16 bit mode for this display
//U8G2_ST75256_JLX240160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
```

```

/* reset= */ 8);
//U8G2_ST75256_JLX240160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST75256_JLX25664_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset= */ 8); // Due, 2nd I2C, enable U8g2 16 bit
mode for this display
//U8G2_NT7534_TG12864R_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_NT7534_TG12864R_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_ST7588_JLX12864_1_SW_I2C u8g2(U8G2_R0, /* clock= */ SCL, /* data= */ SDA, /* reset= */ 5);
//U8G2_ST7588_JLX12864_1_HW_I2C u8g2(U8G2_R0, /* reset= */ 5);
//U8G2_IST3020_ERC19264_1_6800 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable= */ 28, /*cs= */ 32,
/*dc= */ 30, /*reset= */ 31); // Connect WR pin with GND
//U8G2_IST3020_ERC19264_1_8080 u8g2(U8G2_R0, 44, 43, 42, 41, 40, 39, 38, 37, /*enable= */ 29, /*cs= */ 32,
/*dc= */ 30, /*reset= */ 31); // Connect RD pin with 3.3V
//U8G2_IST3020_ERC19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_LC7981_160X80_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 18, /*cs= */ 14, /*dc= */
15, /*reset= */ 16); // Connect RW with GND
//U8G2_LC7981_160X160_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 18, /*cs= */ 14, /*dc= */
15, /*reset= */ 16); // Connect RW with GND
//U8G2_LC7981_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 18, /*cs= */ 14, /*dc= */
15, /*reset= */ 16); // Connect RW with GND
//U8G2_LC7981_240X64_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 18, /*cs= */ 14, /*dc= */
15, /*reset= */ 16); // Connect RW with GND
//U8G2_SED1520_122X32_1 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*dc= */ A0, /*e1= */ A3, /*e2= */ A2, /*
reset= */ A4); // Set R/W to low!
//U8G2_T6963_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr= */ 17, /*cs/ce= */ 14,
/*dc= */ 15, /*reset= */ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_256X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr= */ 17, /*cs/ce= */ 14,
/*dc= */ 15, /*reset= */ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_160X80_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr= */ 17, /*cs/ce= */ 14,
/*dc= */ 15, /*reset= */ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr= */ 17, /*cs/ce= */ 14,
/*dc= */ 15, /*reset= */ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_T6963_128X64_ALT_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable/wr= */ 17, /*cs/ce= */ 14,
/*dc= */ 15, /*reset= */ 16); // Connect RD with +5V, FS0 and FS1 with GND
//U8G2_SED1330_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 17, /*cs= */ 14, /*dc= */
15, /*reset= */ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_SED1330_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 17, /*cs= */ 14, /*dc= */
15, /*reset= */ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_RA8835_NHD_240X128_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 17, /*cs= */ 14,
/*dc= */ 15, /*reset= */ 16); // Connect /RD = E with +5V, enable is /WR = RW, FG with GND, 14=Uno Pin A0
//U8G2_RA8835_NHD_240X128_1_6800 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable= */ 17, /*cs= */ 14,
/*dc= */ 15, /*reset= */ 16); // A0 is dc pin, /WR = RW = GND, enable is /RD = E
//U8G2_UC1601_128X32_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9, /*
reset= */ 8);
//U8G2_UC1601_128X32_1_3W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* reset= */ 8);
//U8G2_UC1604_JLX19264_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_UC1604_JLX19264_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ 8);
//U8G2_UC1608_ERC24064_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1608_ERC240120_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8);
//U8G2_UC1608_240X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9, /*
reset= */ 8); // SW SPI, Due ERC24064-1 Test Setup
//U8G2_UC1610_EA_DOGXL160_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */
9, /* reset= */ U8X8_PIN_NONE);
//U8G2_UC1610_EA_DOGXL160_1_4W_HW_SPI u8g2(U8G2_R0, /* cs= */ 10, /* dc= */ 9, /* reset= */ U8X8_PIN_NONE);
//U8G2_UC1611_EA_DOGM240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset= */ 8); // Due, 2nd I2C, DOGM240 Test Board
//U8G2_UC1611_EA_DOGM240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock= */ 13, /* data= */ 11, /* cs= */ 10, /* dc= */ 9,
/* reset= */ 8); // SW SPI, Due DOGXL240 Test Board
//U8G2_UC1611_EA_DOGXL240_1_2ND_HW_I2C u8g2(U8G2_R0, /* reset= */ 8); // Due, 2nd I2C, DOGXL240 Test Board

```

```

//U8G2_UC1611_EA_D0GXL240_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // SW SPI, Due D0GXL240 Test Board
//U8G2_UC1611_EW50850_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // 240x160, Connect RD/WR1 pin with 3.3V, CS is aktive high
//U8G2_UC1611_CG160160_1_8080 u8g2(U8G2_R0, 8, 9, 10, 11, 4, 5, 6, 7, /*enable=*/ 18, /*cs=*/ 3, /*dc=*/
16, /*reset=*/ 17); // Connect WR1 and CD1 pin with 3.3V, connect CS0 with cs, WR0 with enable, CD with dc
//U8G2_UC1638_160X128_1_4W_HW_SPI u8g2(U8G2_R2, /* cs=*/ 2, /* dc=*/ 3, /* reset=*/ 4); // Not tested
//U8G2_SSD1606_172X72_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // eInk/ePaper Display
//U8G2_SSD1607_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // eInk/ePaper Display, original LUT from embedded artists
//U8G2_SSD1607_GD_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Good Display
//U8G2_SSD1607_WS_200X200_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/
9, /* reset=*/ 8); // Waveshare
//U8G2_IL3820_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9, /*
reset=*/ 8); // WaveShare 2.9 inch eInk/ePaper Display, enable 16 bit mode for this display!
//U8G2_IL3820_V2_296X128_1_4W_SW_SPI u8g2(U8G2_R0, /* clock=*/ 13, /* data=*/ 11, /* cs=*/ 10, /* dc=*/ 9,
/* reset=*/ 8); // ePaper Display, lesser flickering and faster speed, enable 16 bit mode for this
display!
//U8G2_LS013B7DH03_128X128_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS027B7DH01_400X240_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_LS013B7DH05_144X168_1_4W_HW_SPI u8g2(U8G2_R0, /* cs=*/ 10, /* dc=*/ U8X8_PIN_NONE, /* reset=*/ 8);
// there is no DC line for this display
//U8G2_ST7511_AVD_320X240_1_8080 u8g2(U8G2_R0, 13, 11, 2, 3, 4, 5, 6, A4, /*enable/WR=*/ 7, /*cs=*/ 10,
/*dc=*/ 9, /*reset=*/ 8); // Enable U8g2 16Bit Mode and connect RD pin with 3.3V/5V

```

```

// End of constructor list

```

```

#define SUN 0
#define SUN_CLOUD 1
#define CLOUD 2
#define RAIN 3
#define THUNDER 4

```

```

void drawWeatherSymbol(u8g2_uint_t x, u8g2_uint_t y, uint8_t symbol)

```

```

{
    // fonts used:
    // u8g2_font_open_iconic_embedded_6x_t
    // u8g2_font_open_iconic_weather_6x_t
    // encoding values, see: https://github.com/olikraus/u8g2/wiki/fntgrpiconic

```

```

switch(symbol)

```

```

{

```

```

    case SUN:

```

```

        u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);

```

```

        u8g2.drawGlyph(x, y, 69);

```

```

        break;

```

```

    case SUN_CLOUD:

```

```

        u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);

```

```

        u8g2.drawGlyph(x, y, 65);

```

```

        break;

```

```

    case CLOUD:

```

```

        u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);

```

```

        u8g2.drawGlyph(x, y, 64);

```

```

        break;

```

```

    case RAIN:

```

```

        u8g2.setFont(u8g2_font_open_iconic_weather_6x_t);

```

```

        u8g2.drawGlyph(x, y, 67);

```

```

        break;

```

```

    case THUNDER:
        u8g2.setFont(u8g2_font_open_iconic_embedded_6x_t);
        u8g2.drawGlyph(x, y, 67);
        break;
    }
}

void drawWeather(uint8_t symbol, int degree)
{
    //绘制天气符号
    drawWeatherSymbol(0, 48, symbol);
    //绘制温度
    u8g2.setFont(u8g2_font_logisoso32_tf);
    u8g2.setCursor(48+3, 42);
    u8g2.print(degree);
    u8g2.print("°C"); // requires enableUTF8Print()
}

/*
    Draw a string with specified pixel offset.
    The offset can be negative.
    Limitation: The monochrome font with 8 pixel per glyph
*/
void drawScrollString(int16_t offset, const char *s)
{
    static char buf[36]; // should for screen with up to 256 pixel width
    size_t len;
    size_t char_offset = 0;
    u8g2_uint_t dx = 0;
    size_t visible = 0;

    u8g2.setDrawColor(0); // clear the scrolling area
    u8g2.drawBox(0, 49, u8g2.getDisplayWidth()-1, u8g2.getDisplayHeight()-1);
    u8g2.setDrawColor(1); // set the color for the text

    len = strlen(s);
    if ( offset < 0 )
    {
        char_offset = (-offset)/8;
        dx = offset + char_offset*8;
        if ( char_offset >= u8g2.getDisplayWidth()/8 )
            return;
        visible = u8g2.getDisplayWidth()/8-char_offset+1;
        strncpy(buf, s, visible);
        buf[visible] = '\0';
        u8g2.setFont(u8g2_font_8x13_mf);
        u8g2.drawStr(char_offset*8-dx, 62, buf);
    }
    else
    {
        char_offset = offset / 8;
        if ( char_offset >= len )
            return; // nothing visible
        dx = offset - char_offset*8;
        visible = len - char_offset;
        if ( visible > u8g2.getDisplayWidth()/8+1 )
            visible = u8g2.getDisplayWidth()/8+1;
        strncpy(buf, s+char_offset, visible);
        buf[visible] = '\0';
    }
}

```

```

    u8g2.setFont(u8g2_font_8x13_mf);
    u8g2.drawStr(-dx, 62, buf);
}

}

void draw(const char *s, uint8_t symbol, int degree)
{
    int16_t offset = -(int16_t)u8g2.getDisplayWidth();
    int16_t len = strlen(s);

    u8g2.clearBuffer();           // clear the internal memory
    drawWeather(symbol, degree); // draw the icon and degree only once
    for(;;)                       // then do the scrolling
    {

        drawScrollString(offset, s); // no clearBuffer required, screen will be partially cleared here
        u8g2.sendBuffer();           // transfer internal memory to the display

        delay(20);
        offset+=2;
        if ( offset > len*8+1 )
            break;
    }
}

void setup(void) {

    /* U8g2 Project: SSD1306 Test Board */
    pinMode(10, OUTPUT);
    pinMode(9, OUTPUT);

    digitalWrite(10, 0);
    digitalWrite(9, 0);

    /* U8g2 Project: T6963 Test Board */
    //pinMode(18, OUTPUT);
    //digitalWrite(18, 1);

    /* U8g2 Project: KS0108 Test Board */
    //pinMode(16, OUTPUT);
    //digitalWrite(16, 0);

    /* U8g2 Project: LC7981 Test Board, connect RW to GND */
    //pinMode(17, OUTPUT);
    //digitalWrite(17, 0);

    /* U8g2 Project: Pax Instruments Shield: Enable Backlight */
    //pinMode(6, OUTPUT);
    //digitalWrite(6, 0);

    u8g2.begin();
    u8g2.enableUTF8Print();
}

void loop(void) {
    draw("What a beautiful day!", SUN, 27);
    draw("The sun's come out!", SUN_CLOUD, 19);
    draw("It's raining cats and dogs.", RAIN, 8);
    draw("That sounds like thunder.", THUNDER, 12);
}

```

```
draw("It's stopped raining", CLOUD, 15);  
}
```

眼尖的读者可以看出，无论是Mega还是NodeMcu，它的代码完全是一样，果然这个库真的很强大！！

实验结果：跟Mega测试效果一样，略图。

6.总结

博主大体上介绍了这个强大库U8G2 OLED库，但是博主需要提醒的是，中文字库非常大，比较耗内存，谨慎使用。如果只是显示一些固定的中文，建议自行获得汉字编码，通过drawXBM的方法去显示。

以下为博客版权说明，拒绝盗版，未经允许，请勿商用：

交流免费qq群（[点我一键加群](#)）：



群名称:ESP8266物联网开发之旅
群 号:834493093

- 优点：主要是营造技术交流平台，基于博哥目前分享开源的帖子，群里也有来自于各方的求学者
- 缺点：属于自学群，博哥不一定有时间回答问题

交流付费qq群（[点我一键加群](#) 入群15rmb）：



群名称:物联网开发之旅VIP
群 号:491507716

- 优点1：不定期推送干货，拒绝伸手党，提高群员素质。最新教程或者app率先在此群更新，开放apk以及源码，第一时间回复群问题。所得收入博哥会用于管理群以及购买其他硬件设备，不断推出其他教程
- 优点2：帖子整理成pdf文档，方便下载，同时群文件会有丰富资源文件，可供初学者学习查阅

博哥私人号：

- 用途：与博哥的商业合作（付费教学、付费项目、毕业设计、项目合作或者其他商业合作）
- 个人技能：Android App开发、小程序系列开发、单片机开发等
- 个人履历：均工作于广州大型公司，独立设计App，也带领过小团队

单片机菜鸟博哥创作 版权声明